Smart Motorcycle Crash Detection Helmet

Crest Gold Award

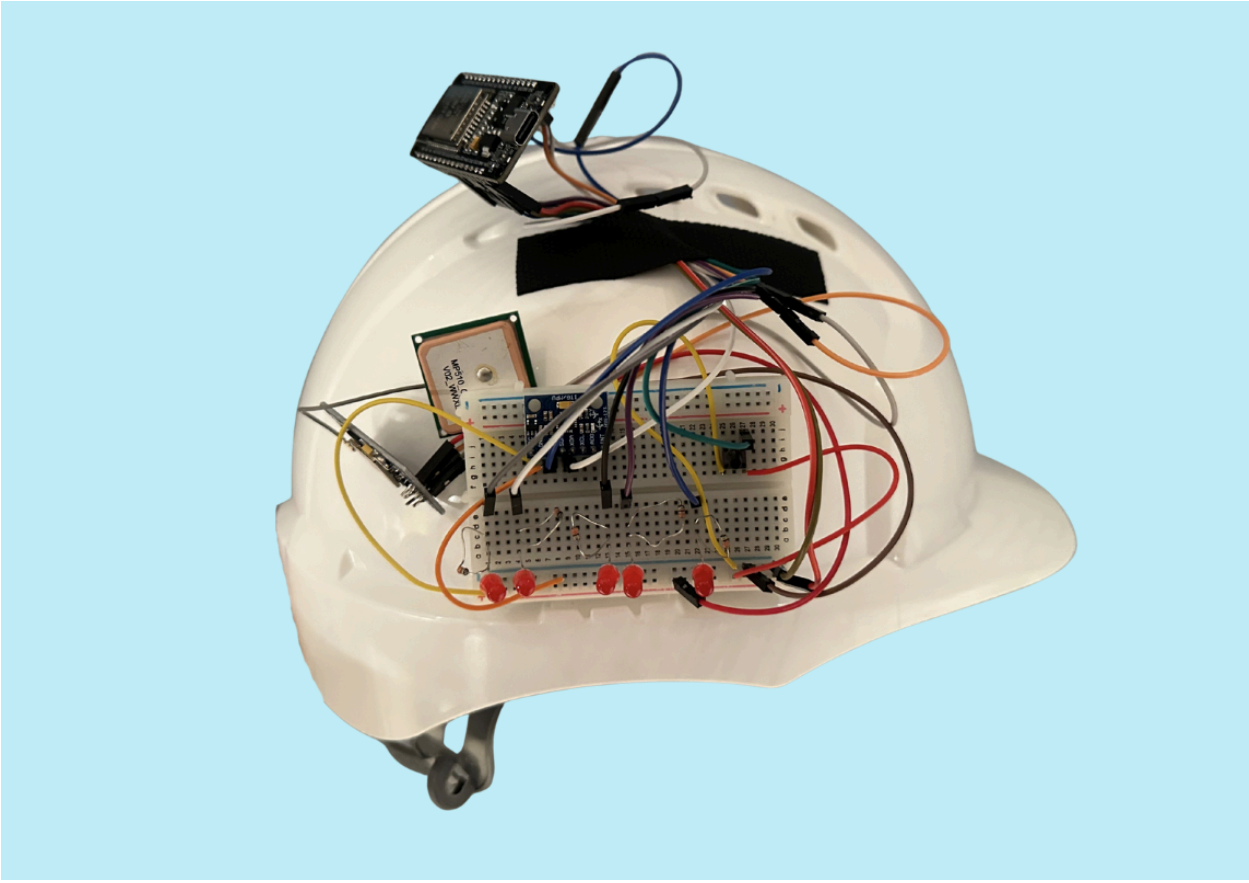


Table of Contents

1. Introduction **3**
1.1 Abstract 3
1.2 Problem and Objectives 3
1.3 Time Management 4
1.4 Materials and People 4
2. Background Research **6**
2.1 Existing Products 6
2.2 Areas for Improvement 8
2.3 Research Questions 8
2.4 Market Gap 9
2.5 Component Research 9
2.6 Phone-based vs Helmet-based Systems 11
2.7 Simple Threshold System vs Multi-Feature Classification 12
2.8 Comparing Connectivities 12
3. Design **15**
3.1 Additional Features 15
3.2 Helmet Design 15
3.3 Proposed Solution 18
4. Safety and Ethics **19**
5. Planning the Crash Detection Algorithm **22**
5.1 Proposed System Architecture 22
5.2 Program Structure 23
6. Blynk Development **25**
7. Sensor Calibration **28**
8. Testing **31**
9. Results **34**
9.1 Results of Testing 34
9.2 Graphs 36
10. Feature Selection **45**
10.1 Selecting Parameters 45
10.2 Standard Deviation 45
10.3 99th Percentile 45
10.4 Alternative Parameters 46
11. Writing the Crash Detection Algorithm **47**
11.1 Overview 47
11.2 Rolling Window 48
11.3 Algorithm Classification Conditions 49
11.4 Updating the Blynk Dashboard 50
11.5 Final System Validation 51
12. Problems Encountered **53**
13. Limitations **55**
14. Wider Implications **57**
15. Evaluation **59**
15.1 Skills Gained 59
15.2 What I Would do Better 59

16. Conclusion **61**
16.1 Comparison against project objectives 61
16.2 Technical Findings 63
16.3 Future Improvements 64
17. Bibliography **65**

1. Introduction

1.1 Abstract

For my Crest Gold Award, I made a smart motorcycle helmet that uses acceleration and gyroscope values to detect an accident and automatically sends an alert to emergency services and emergency contacts with the location of the rider. I managed to create a working prototype that reliably detects impact-like events and thus sends an email to my phone. The finished product serves as a strong foundation for future developments I will make, with the hope of my work reaching the standard needed for real-world application. During testing I deliberately did not connect the system directly to 999, to prevent accidental emergency calls. However, the communication architecture was designed so that additional authorised recipients, including an approved emergency-service interface, could be incorporated if the system were developed into a deployable product.

1.2 Problem and Objectives

In 2025, 15,447[14] motorcyclists were reported as casualties in road collisions in Great Britain, including 386 fatalities. Unlike cars, most motorcycles do not have an in-built SOS calling system, meaning that crashes leave riders unable to call for help. The aim of this project is to increase rider safety by decreasing the amount of time between the impact-event and emergency services being alerted.

For my final product to be successful, it should accomplish the following objectives:

1. Data Collection
    - 1.1. Constantly collect accelerometer and gyroscope data from the helmet
    - 1.2. Process the data and calculate statistical values for the samples
2. Product Reliability
    - 2.1. Calibrate sensor readings such that the data I collect is accurate and representative of the current situation
3. Crash Detection
    - 3.1. Distinguish normal riding from crash-like events using accelerometer and gyroscope data
    - 3.2. Identify scenarios which could create false crash alarms and create measurements to reduce nuisance alerts
    - 3.3. Develop a multi-stage crash detection algorithm that relies on multiple factors
    - 3.4. Trigger an automated emergency alert and location message using the Internet of Things, with provision for future authorised emergency-service integration.
4. User Interaction
    - 4.1. Provide the user with a method of cancelling the emergency alert if they are safe

1.3 Time Management

Before starting the project, I planned out what the main milestones were and how I was going to use my time to accomplish them. The most effective way was to use a Gantt Chart (***Figure 1*** below), as this allowed me to plan everything out in terms of weeks and the hours each task was expected to take. For the aspects that I knew would take me the longest (research, gathering data and writing the algorithm), I made sure to give them enough weeks to account for any delays. Using the Gantt Chart will help me stay on track with the project as I can check if I am on schedule or not. The planned project total was 81 hours, exceeding the approximately 70 hours expected for the Crest Gold Award.

TASK	HOURS	01-Jun	08-Jun	15-Jun	22-Jun	29-Jun	06-Jul	13-Jul	20-Jul	27-Jul
Research	15	x	x	x						
Sensor Learning	5				x	x				
Calibration of Sensor	8						x	x		
Data Collection	15								x	x
Data Analysis	7									
Planning Algorithm	2									
Writing Code	20									
Helmet Assembly	2									
Blynk Integration	2									
Algorithm Testing	5									
Total Hours	81									
Key:	In Progress	Finished								

TASK	HOURS	03-Aug	10-Aug	17-Aug	24-Aug	31-Aug	07-Sep	14-Sep	21-Sep	28-Sep	05-Oct
Research	15										
Sensor Learning	5										
Calibration of Sensor	8										
Data Collection	15										
Data Analysis	7	x									
Planning Algorithm	2		x	x							
Writing Code	20				x	x	x	x			
Helmet Assembly	2								x		
Blynk Integration	2								x		
Algorithm Testing	5									x	x
Total Hours	81										
Key:	In Progress	Finished									

Figure 1
Gantt Chart

1.4 Materials and People

Throughout this project, I worked with a mentor who guided me and gave me advice. Being able to learn from someone working in the industry was incredibly useful, especially when planning the testing regime and analysing the results collected. While I implemented, tested and developed the crash-detection algorithm using my experimental results, my mentor helped me identify errors and debug the program.

I was able to borrow most of the components I needed from school, but had to order the ESP32, MPU 6050, and GPS module off Amazon. While doing this, I ensured the products were reviewed and that they were not faulty.

One of the biggest limitations with equipment was the lack of a motorcycle. As a solution, I carried out most of the motion tests in a car with help from my parents. Unfortunately, this also meant that I was unable to test specific movements, such as turning by leaning, which is different in a car than it is on a motorcycle.

2. Background Research

2.1 Existing Products

The first product I came across was Quin[1] - a crash detection system made up of a physical pod that clicks into place on a bicycle helmet, a ‘Quin tag’ and an app (***Figure 2*** below). The system detects dangerous crashes and how severe they are, as well as automatically calls for help. The ‘Quin tag’ acts as a digital medical profile; anybody can scan the tag and see important information about the rider. This is crucial in the first response to a crash, as it gives medical officers crucial information needed to carry out treatment. The device can also be used with a motorcycle helmet, and the company is releasing a helmet with this crash detection system built into it. Aside from being a safety device, it is also a general bike fitness tracker.

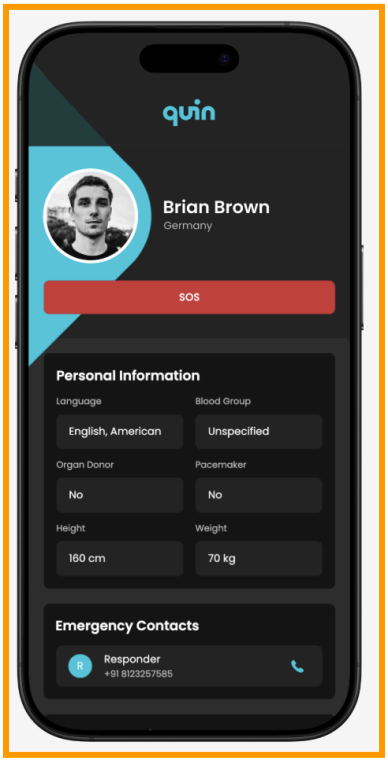


Figure 2[1]
Image of app display when Quin tag is scanned

The second motorcycle crash detection program I found was Realrider SOS[2] - a paid app that users can install on their phone (***Figure 3*** below). This system has a direct line to 999 and uses sensors to assess rider movement and thus crashes. Though this means no additional sensor is needed or a specific type of helmet, a key limitation to this is that the program relies on the user’s phone. If the phone gets damaged within the crash, the app can no longer run. As well as

this, even forgetting your phone or it running out of battery means that the app cannot monitor your ride.

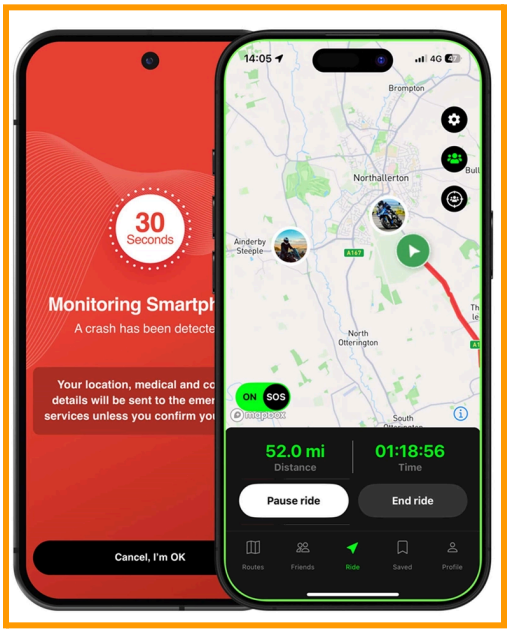


Figure 3[3]
Image of the RealRider SOS interface

The third product that I came across was the Tocsen Sensor[4] - a helmet add-on (***Figure 4***). The sensor notifies emergency contacts and contacts the local rescue community sending the rider's location as well. The product also has the capability to alert the user if other Tocsen users are in the area, and has the option for the user to switch off the sensor or put it into stand-by mode if in a group. Knowing when there are other users near-by can be very useful if riders need someone to help them.



Figure 4[4]
Image of the Tocsen Sensor

2.2 Areas for Improvement

All three of the products researched rely on a separate sensor that is not yet built into the helmet itself or a mobile phone. These individual devices have a higher risk of getting damaged themselves during the crash, as they do not make use of the protection a helmet offers. As such, for my product, I will look at ways of incorporating the sensor within the helmet itself, if possible, so that it is shielded better and so that it reduces the need for the user to remember to put the sensor on the helmet. A key area most crash detection devices struggle with is false alert prevention - where a rapid but safe motion is categorised as a collision. One way I plan on improving this is by incorporating some method for checking movement after a crash; this could potentially take the form of a button that the user can press to confirm that they are ok.

2.3 Research Questions

To help me understand the main considerations of the project, I created the following series of research questions to answer to provide more clarity onto the project and how I will go about it.

1. *How to distinguish between a crash and normal riding movement?*

Normal riding will have small bumps but smooth turns[5], and by creating motion thresholds and looking out for a g-force spike to indicate a sudden hit, I could potentially detect a crash. I could also check for any post-event motion, as a crash would not have any.

2. *What could be some conditions where I could see false alerts?*

Dropping the helmet, going over a speed bump very fast, harsh braking, potholes, hairpin turns.

3. *How would I work on these?*

Create safe replicas of each event to test each scenario and see what sensor values each of them give. I could then be able to either increase the crash thresholds or specifically categorise those values as not a crash. Perhaps if one of these events were detected, I could also wait for the user to confirm that they are safe. Below are each of the scenarios and the possible ways I could differentiate them from a crash.

- Dropping the helmet: GPS values won't change, in a crash they would
- Speed bump: the motorcycle would slow down first then increase
- Harsh braking: a microphone could be used to detect background sounds
- Potholes: same as harsh braking
- Hairpin turn: the motion of the bike would eventually stabilise but in a crash it would not

4. *What thresholds should be used for the gyroscope / accelerometer?*

It is important to separate the use of a gyroscope from an accelerometer. The former is used to measure angular velocity, which is the rate of change of rotation. The latter is used to measure the rate of change of velocity. Getting the exact values will come through testing, so I can't determine them exactly now, but from my research I have seen that the g-force exerted during a crash could be from around 40Gs to over 100Gs[6]. It is also important to remember however, that during a crash, the rider will most likely separate from the bike and slide, which is why all the sensors will be on the helmet rather than on the motorcycle itself.

5. *What are the most common crash types?*

- Low slide[7]: the bike loses traction in a turn and slides out from under the rider
- High slide[8]: the rear tire loses traction, slides sideways and then regains grip
- Tank slapper post wheelie[8]: the front wheel touches down at an angle causing the motorbike to lose balance.

2.4 Market Gap

The safety features which currently call emergency services after a crash are restricted mainly to cars. Helmets need to be replaced around every 3-5 years[9], and with electronics, people may not want to buy a brand new expensive helmet every couple of years. As such could I make my product such that the circuitry is part of the helmet but can be removed and placed onto another one? Though more and more crash detection products are being created, a key part where lots of them fail is within nuisance alerts / false alarms. To make my smart motorcycle helmet stand out, I will focus on ensuring accuracy of crash detection. One feature that really stood out to me while conducting this background research was the Quin tag. Having some sort of QR code that can be scanned to provide first responders with the patient's medical history would be extremely helpful.

2.5 Component Research

For this project, the main components I will need are the following:

- Some type of microcontroller
- Accelerometer
- Gyroscope
- GPS module
- Push button

I have mainly used Arduino before with my projects, but when I was researching other alternatives, I came across the ESP32. Since I will use the Internet of Things to call emergency services, the ESP32 works quite well as it is used for IoT specifically as it has built-in WIFI[10].

Another useful feature is that I can still use the Arduino IDE, saving me time in having to learn a new programming format. The module also has sleep modes for battery saving - something which I could enable when the helmet is not in use.

I initially came across the ESP32 through tutorials, but I then checked the manufacturer's official technical documentation to understand whether it was actually suitable for the project. Espressif describes the ESP32 as a 32-bit microcontroller with built-in 2.4 GHz Wi-Fi and Bluetooth connectivity. It also has multiple GPIO pins as well as I²C and UART interfaces. These interfaces are particularly useful for my project because I can communicate with the MPU6050 through I²C while using one of the UART interfaces for the GPS module.The ESP32 also has several power-saving modes including Light-sleep and Deep-sleep. Although I did not fully implement power management in this prototype, this could become important in a later battery-powered version of the helmet. Most importantly, the built-in Wi-Fi means I can connect to Blynk without having to add a separate Wi-Fi module, reducing the size and complexity of the prototype.

When doing research on accelerometers and gyroscopes, I came across the MPU 6050 - a combined 3-axis accelerometer and 3-axis gyroscope. I think this is better than having separate sensors for each of the two functions, as it reduces the wiring and thus potential for the circuit to break as well as the physical size of the system. The component contains an IMU (inertial measurement unit). Its balance and angular velocity is measured by the gyroscope, and its linear acceleration is measured by the accelerometer. It is important to note that even when the helmet is stationary, gravity will still be acting. This means that the acceleration in one of the planes will still be approximately $10ms^{-2}$. Whether this is in the x, y, or z plane depends on the orientation of the sensor. According to the manufacturer's product specification, the accelerometer has selectable measurement ranges of ±2 g, ±4 g, ±8 g and ±16 g, while the gyroscope can be configured to ±250, ±500, ±1000 or ±2000 degrees per second. The MPU6050 communicates with a microcontroller using I²C.

Having both sensors in one component was useful because it reduced the number of separate devices and wires required on the helmet. I eventually configured the accelerometer to ±16 g and the gyroscope to ±2000°/s because these were the largest available ranges and were therefore more appropriate for the high-motion events being investigated.

This research also revealed an important limitation. Even at its maximum setting, the accelerometer can only directly measure up to ±16 g on each axis. Therefore, a much more severe real motorcycle impact could potentially saturate the accelerometer. The MPU6050 was suitable for developing and testing my prototype, but a future safety system should investigate a dedicated high-g accelerometer.

For the location part of the helmet I used a NEO-6-series GPS module. The manufacturer's documentation shows that the NEO-6 modules support UART communication as well as USB, SPI and an I²C-compatible interface. UART was the most suitable option for this project because the ESP32 has hardware UART interfaces available, allowing the GPS receiver to continuously send positioning information while the MPU6050 uses I²C separately.

The GPS is not used to decide whether an event is a crash. Its role is instead to provide latitude and longitude after an emergency has been confirmed. Separating these jobs makes the crash detector less dependent on GPS availability. During practical testing I also discovered that obtaining a satellite fix was much more reliable outdoors than indoors, so the program stores the most recently received valid coordinates rather than waiting until after an impact to begin collecting location data.

One limitation identified from the manufacturer's information is that the NEO-6 series is now an older-generation product. If the helmet were developed further, I would investigate a newer GNSS module with improved positioning performance and continued manufacturer support.

2.6 Phone-based vs Helmet-based Systems

A key question I had to ask before deciding on this project was the difference between a phone-based safety device and a helmet-based one. Each of these have their advantages and disadvantages that I took into account.

Phone-based Advantages:

- The rider does not need to buy any extra equipment, simply download an app on your phone
- Most people will always have their phones on them when riding the motorcycle, so the safety feature is less likely to be forgotten
- Takes up no physical space for the rider

Phone-based Disadvantages:

- Phone could easily get smashed during a motorcycle accident which would mean the safety app stops working
- If the phone is out of battery, the crash detection will no longer work
- App dependent on the quality of the phone’s sensors. This means that the product could be less effective on older / damaged phones
- Phone may be in a pocket or in a bag, so its motion may not match that of the rider

Helmet-based Advantages:

- Crash detection system is more protected as it could be built into the helmet
- Uses specialised sensors that may be more accurate than those already built into a phone
- Can work independent of a phone
- System will be placed on the rider’s head, so the data gathered is more indicative of their motion

Helmet-based Disadvantages:

- Requires extra electronics and mounting
- More expensive due to components and circuitry

- Introduces the issue of balancing battery life with size

Since this device is first and foremost a safety product, as well as the fact that one of my aims with this project is to create a device that minimises false alarms, accuracy of data is incredibly important. One of the main issues with a phone-based system is that if it is in a bag or a pocket, the acceleration and gyroscope values measured by the phone's sensors may not be reflective of the motion of the rider. This could lead to misclassification of events including the possibility that an actual crash is categorised as safe. Therefore, since this problem does not apply to a helmet-based system, for this particular project, a helmet-based system is better, and thus is what I will be using.

2.7 Simple Threshold System vs Multi-Feature Classification

When making the crash detection algorithm, there are two main approaches: using a singular threshold, or using a multi-feature classifier. The latter would still use threshold values to determine the type of event but for an event to be categorised as a crash, it would need to pass multiple checks. A really important point that I came across during my research is that 'the vehicle dynamics vary according to the nature of each accident'[14]. This means that using a single threshold value for acceleration and gyroscope readings may fail to detect certain crashes.

However, using a single threshold system would be far more simple to implement, as well as would be less computationally heavy. Speed is very important in this device, as one of its uses is to reduce the time taken to call for emergency help after an accident. At the same time, accuracy of the algorithm must also be prioritised. With a simple threshold approach, reliability of the algorithm will be compromised.

Both approaches have their pros and cons, and to ensure that my product balances speed with accuracy, I have decided to go with the multi-feature classification system using mainly the core features that distinguish between normal riding and a crash-like event.

2.8 Comparing Connectivities

For this project, I will use the Internet of Things platform Blynk to allow control of the ESP32 with my phone. Blynk communicates with the ESP32 through Wi-Fi and the Blynk cloud. I was previously unaware of this platform, and was directed to it by my mentor. What stood out to me the most was the simplicity and breadth of features in the program. It was very easy to set up different datastreams and customise the dashboard based on the information I wanted to present. I was also able to set up an interface on my phone as well, which is especially useful.

Blynk is an IoT platform designed to connect hardware devices to cloud-based dashboards and mobile applications. Its system uses datastreams to transfer values between the device and Blynk Cloud, while Virtual Pins allow software-generated values rather than only physical GPIO states to be displayed. This makes it suitable for showing the accelerometer, gyroscope, GPS and algorithm-status values produced by my ESP32.

Another feature that was particularly relevant to this project was Blynk Events. Events can be triggered from the firmware and can then generate notifications for selected users. This gave me a straightforward way to turn an impact detected by the ESP32 into an emergency notification without having to create a mobile application and cloud server myself.

Blynk supports several communication methods, including Wi-Fi and cellular connectivity. For the prototype, I selected Wi-Fi because the ESP32 already contains Wi-Fi hardware, meaning that no additional communications module was required. Blynk also supports cellular-connected hardware (e.g a GSM module), so a later version could use a cellular modem and mobile network instead. Each of these approaches have their advantages and disadvantages which are broken down below.

WiFi + Blynk Advantages:

- Experimental research into a Blynk-based system found that both WiFi and GSM successfully transmit data with WiFi having a lower latency [15]
- No additional hardware needed
- Easy to set up dashboard and notifications with the Blynk interface

WiFi + Blynk Disadvantages:

- Dependent on a WiFi connection / hotspot
- This is not suitable if the rider is moving between location, where the WiFi network will change

Blynk + GSM Advantages:

- Works over a much larger geographical area
- The rider will not need access to a WiFi hotspot
- The Blynk console and interface can still be used

Blynk + GSM Disadvantages:

- Requires extra GSM module which increases complexity of the system
- Still needs to be in an area with mobile signal
- Increases the power demand of the system, resulting in lower battery life
- Increases cost of system

Blynk + GSM has the greatest flexibility in the sense that it is not dependent on a WiFi connection, and can send messages in more locations. This is very important, as it increases the reliability of the emergency alert being transmitted, which is integral in calling for help. However, the combination of WiFi + Blynk is the simplest to implement, and the fact that no external component is needed, will help reduce the size of the physical system. For my

prototype, Wi-Fi + Blynk was the most suitable choice because Wi-Fi was already built into the ESP32 and Blynk provided both a dashboard and an event-notification system without additional communication hardware. However, my research also showed that this solution depends on access to a suitable Wi-Fi network. For a real motorcycle safety product, as stated earlier, cellular connectivity or a system with multiple communication methods would be more appropriate because the rider cannot be expected to remain connected to the same Wi-Fi network while travelling. Therefore, Wi-Fi + Blynk was selected as a prototyping solution rather than assumed to be the final commercial communication architecture.

3. Design

3.1 Additional Features

From the background research I conducted, I started brainstorming potential extra features to put on the helmet. As discussed in the previous section, I am really keen on adding some sort of QR code that when scanned can display the rider's emergency health information but I am not sure how feasible this would be for the purposes of my prototype.

As suggested previously in the document, another feature to implement would be checking rider condition after a crash. If there was a false alarm, or if the rider is completely fine and only a small incident occurred, they could click on a button on the bottom of the helmet. This would prevent emergency services from being called unnecessarily. Using a button for this function may not be the best solution since it could get accidentally pressed if the helmet hits the ground during the collision, so the position of this button is very important.

Another additional feature could be to have automated lights after a crash, which would help riders to be spotted at night. These would be turned on after the alert has been sent to emergency services.

3.2 Helmet Design

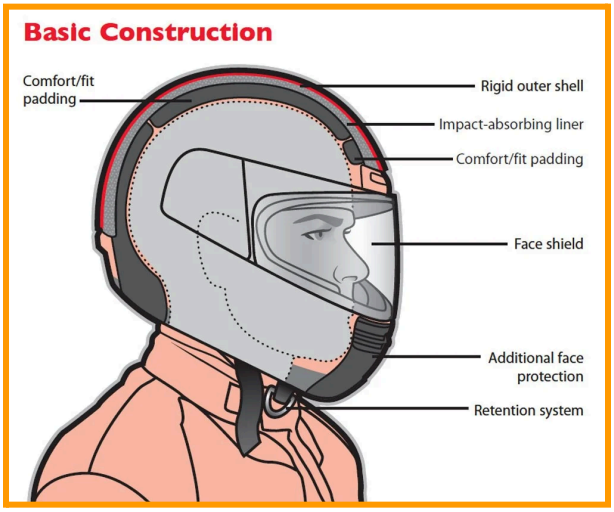


Figure 5
Image of a helmet design[11]

My plan was to house all of the electronics within the helmet itself, but as seen in ***Figure 5*** above, all helmets have a rigid outer shell followed by an impact-absorbing liner. Drilling into this structure would compromise its integrity, and putting anything inside the foam padding would reduce the protection offered. I then started thinking about the spaces in a helmet (as seen in pink in ***Figure 5***), however, accessing them would result in these same problems. Putting the

circuitry next to the person's head is also not an option, and this could lead to serious injury during a crash. Therefore, for the purposes of the prototype, unfortunately, the main viable option is to put all the electronics within a small box that can fit onto the outside. This way, the system would still be removable, but I will need to think carefully about protecting the equipment.

Since the outer shell of the helmet cannot be drilled into, I have to think about how the electronics will actually be attached. I will probably house all of the electronics in a casing which can then be put onto the helmet, for which I found 2 different ways.

1. Chin attachment



Figure 6
Amazon Link to Attachment

I have seen many people use this chin attachment (***Figure 6*** above) for Go Pro attachments, and so the method has been tried and tested. Some mounts are screw tightened, so are easy to remove or replace. However, one concern is that if the helmet is dropped, or if the rider falls forwards, then the electronics casing would be the first point of contact and could thus easily be broken. Furthermore, since the back plate of the attachment (***Figure 7*** below) goes inside the helmet, during a crash this could potentially hit the rider.



Figure 7
Back plate of chin attachment

2. Go Pro Adhesive Mount



Figure 8
Amazon Link to Mount

This adhesive mount (***Figure 8*** above) has a sliding mechanism to lock the device in place. I could design the electronics container in such a way that it is able to slide into the pre-existing grooves. Even though simply using adhesive seems a bit risky, such mounts are used very often for Go-Pros, which are heavier than what the electronics will be. Furthermore, this provides a more versatile solution to the chin attachment, as with adhesive, the electronics could be attached on the front, top or side of the helmet.

To ensure the safety of the electronics, I did some research to see if there was a particular location on a helmet where there is the least likelihood of damage. I found an article which provided me with ***Figures 9 and 10*** below.

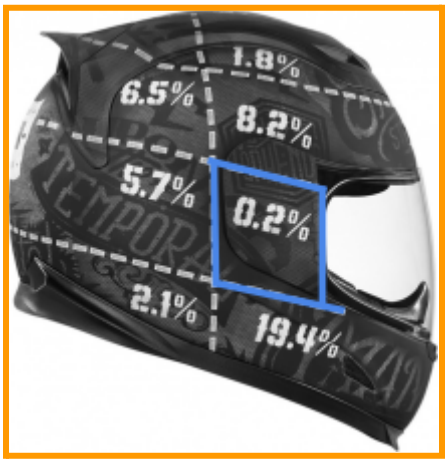


Figure 9[12]
Image of the left side of a helmet and the likelihood of different sections getting damaged



Figure 10[12]

Image of the right side of a helmet and the likelihood of different sections getting damaged

From the figures above, we can see that the section outlined in blue (left cheek) in ***Figure 9*** has the overall lowest probability of getting damaged - 0.2%. Therefore, it follows that this will be the best place to keep the electronics.

3.3 Proposed Solution

Given the research and the design restrictions encountered, my solution proposes that all the circuitry is stored in one box, which can slide into place on the Go Pro adhesive mounts. Keeping the electronics on the side of the helmet not only helps ensure its protection, but it is also easy for the riders to spot if they have forgotten to put the sensor back on, unlike putting the electronics at the back of the helmet.

4. Safety and Ethics

Before carrying out the tests, I conducted a risk assessment of the project. The table below (***Table 1***) assesses both ethical and safety issues, as well as lists solutions to the risks posed. Storing user data must be done in a secure way to ensure data integrity and user privacy.

I assessed both risks created by carrying out the project and risks that could arise if the prototype itself failed. I considered false positives, false negatives, loss of communication and privacy alongside the physical risks of impact testing. Since this is a prototype rather than a certified safety device, it was never relied upon for real emergency protection.

Table 1: *Risk Assessment Matrix*

Hazard	**Risk**	**Control**
Dropping / throwing the helmet during testing	Injury caused by the impact of the helmet	- Clear the test area - Heights are controlled and sensible - Test is carried out under adult supervision
Pendulum test	Helmet rebounding and striking back	- Stay away from path of pendulum - Keep a fixed release position
Road testing	Distraction to the driver	- The student (me) is the passenger - An adult is the driver - There is no interaction by the driver
Electronics	Short circuit and thus electric shock or fire	- Use insulated wiring - Connect circuit to a low voltage supply
Disturbance tests	Damage to the helmet	- Use a construction helmet rather than an existing motorcycle helmet to prevent damage to the structure thus compromising any future use
False positive alarms	Unnecessary concern or unnecessary emergency	- Implement a feature that allows riders to

	notification	confirm they are OK - The alert is sent only if no confirmation is received
False-negative crash detection	A genuine crash might not produce an alert, creating false confidence in the prototype	- Clearly label system as an experimental prototype - Do not depend on it for real emergency protection - Use multiple acceleration/gyroscope features rather than one threshold - Document need for larger independent validation before real use
Live GPS fix	Access to rider's location could compromise privacy	- Restrict access to the location of the rider to the recipient of the emergency alert
Storing emergency contacts	Personal information could be exposed	- Obtain consent before storing someone's phone number or email - Ensure that when contacts are updated, previous data is deleted
Loose USB leads or jumper wires during testing	Wires could snag, disconnect or cause the wearer to trip	- Secure loose wires before tests - Keep cables away from walking paths - Do not conduct unrestricted riding tests while tethered to a laptop/power supply
Wi-Fi/Blynk connection failure	Remote emergency notification may not be delivered	- Check connection during prototype tests - GPS/communication status displayed on dashboard - System not relied on for real safety - Future design to use

		cellular communication and connectivity failover
Publishing firmware containing Blynk/Wi-Fi credentials	Unauthorised access to network or Blynk device	- Replace all real passwords/tokens with placeholders in the submitted Appendix - Regenerate exposed Blynk token - Never publish private credentials

5. Planning the Crash Detection Algorithm

5.1 Proposed System Architecture

Having now decided on the helmet design, I need to focus on the circuitry and how everything will connect. This can be broken down into the structural and virtual design, with the latter referring to how the code will be set up.

Structural Architecture:

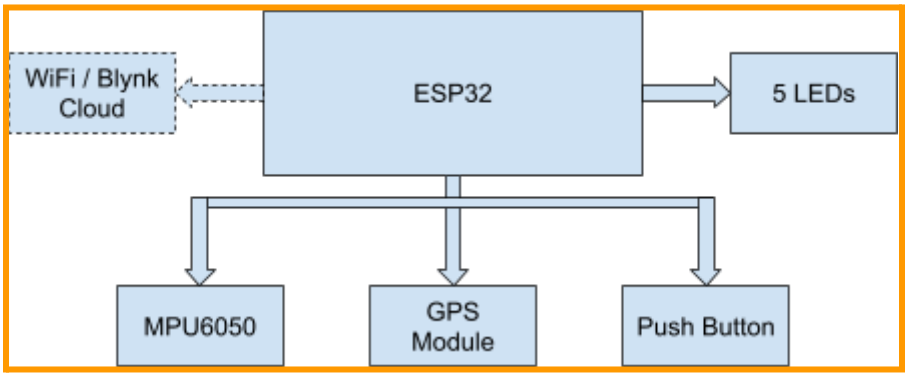


Figure 11
Electronics Architecture

There will be 3 components directly connected to the ESP32 - the MPU 6050, a GPS module, and a push button (***Figure 11*** above).

Virtual Architecture:

The main part of the software will be the crash detection algorithm, however, for optimum functionality of the helmet, the software cannot stop here. There will need to be some user input to the system that acts as confirmation of the crash. A general overview of the code lay out can be seen below in ***Figure 12***.

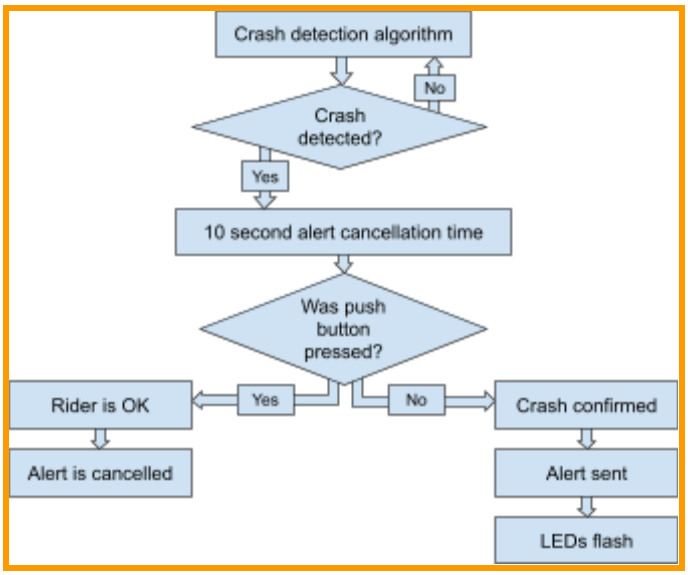


Figure 12
Code Structure

The crash detection algorithm can also be broken down further into sub-tasks, which are outlined below. Using decomposition here will help keep the code organised and will make it easier to debug when I encounter problems.

1. Read accelerometer values in the X, Y and Z plane
2. Calculate total acceleration using the formula $\sqrt{(X^2 + Y^2 + Z^2)}$
3. Read gyroscope values in the X, Y and Z plane
4. Calculate total gyroscope using the formula $\sqrt{(X^2 + Y^2 + Z^2)}$
5. Calculate statistical features for both acceleration and gyroscope
6. Compare these features against the dataset built from the testing
7. Classify current motion of the helmet into normal, disturbance or impact-like

5.2 Program Structure

To maintain good coding practise, I will layout my code in the following manner:

1. Installation of libraries
    - This will include the Wifi libraries and the library that allows the ESP32 to connect to Blynk
2. Configuration of variables
    - The most important variables will be the Wifi name, password and the Blynk authentication token, as these are what facilitates the actual connection between the ESP32 and the Internet of Things
    - Additional variables will be added to prevent the use of magic numbers within my code

3. Global objects
    - Global objects are those which I will use all throughout my code. One such example will be the timer.
4. Functions
    - This is where I will define all my custom functions
    - I will follow the single responsibility principle so that each function only has one responsibility. This once again makes debugging the code easier.
5. Setup
    - As with any Arduino code, the setup look only occurs once at the beginning of the program's execution
6. Main loop
    - This loop will continuously run until the program is stopped

6. Blynk Development

Since this is my first time using both Blynk and an ESP32, I spent some time creating and testing practice programmes to grow my confidence and skill. For these tests, I simply connected the ESP32 to my laptop (***Figure 13*** below) using a USB-C data cable and was able to program on the Arudino IDE.

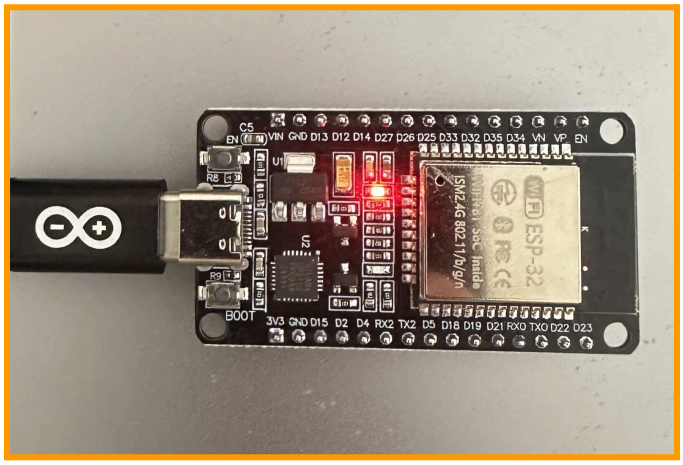


Figure 13
ESP32 connected to my laptop

The very first program (***Figure 15*** below) I ran was setting up the ESP32 and its connection with Blynk. This was relatively straightforward, and laid the crucial foundation to develop my code further. The code creates a counter that is updated and displayed on my Blynk dashboard every second (***Figure 14*** below).

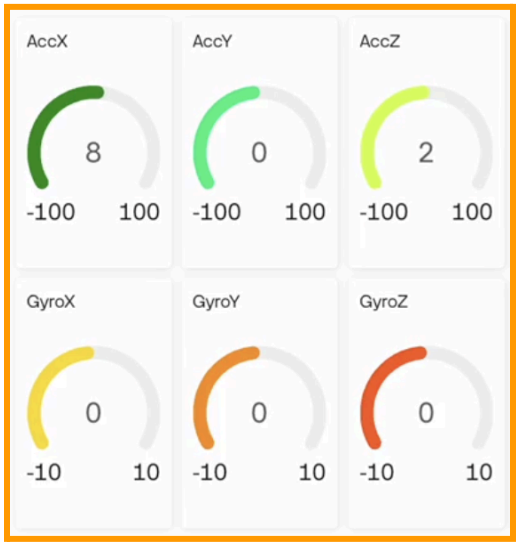


Figure 14
Blynk dashboard updating

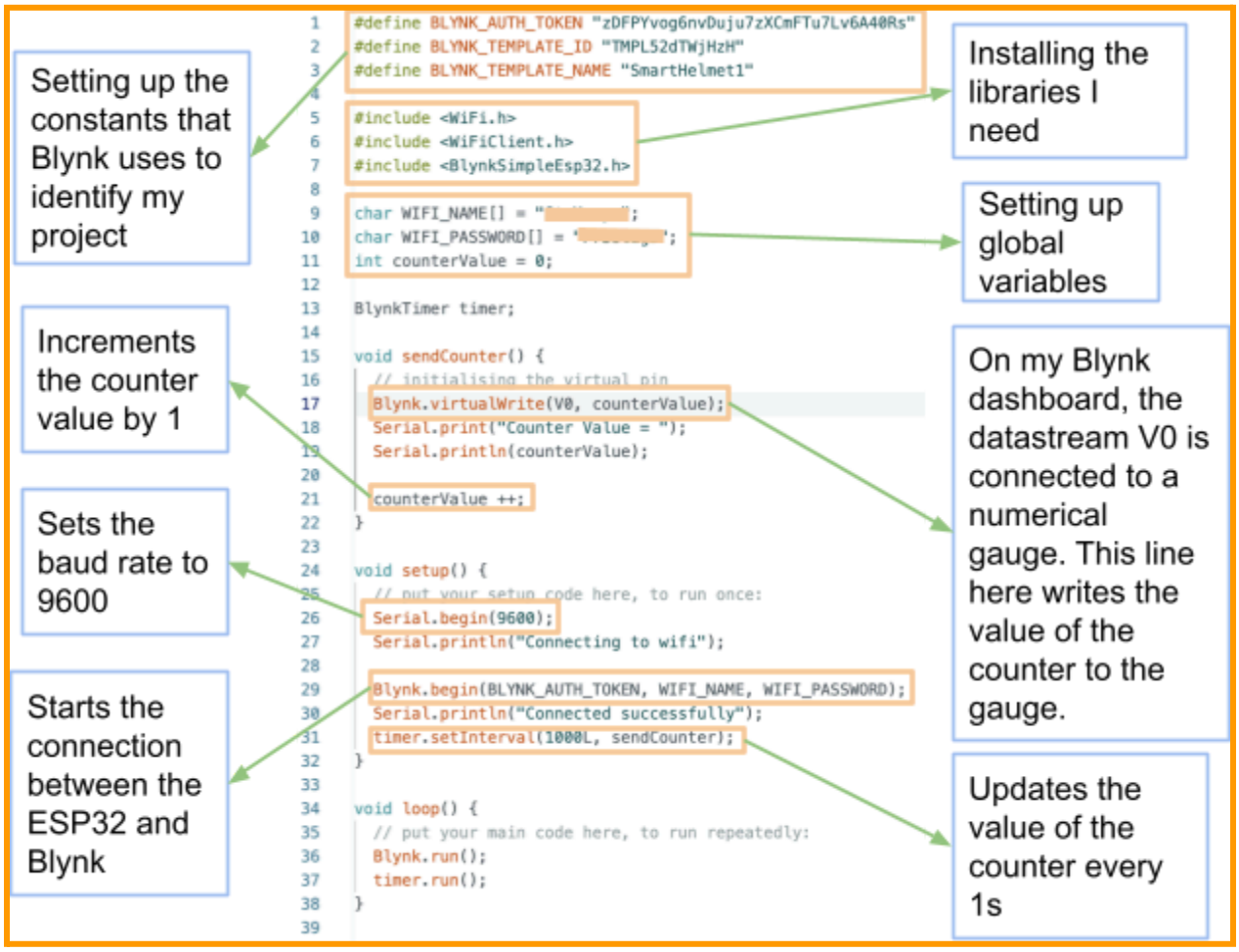


Figure 15
ESP32-Blynk set-up code

To further practise my coding, my mentor then set me the task of changing the code so that Blynk updates every 2 seconds and then every 0.5 seconds. This was simply done by changing the interval defined in the **timer.setInterval** line. The new lines of code can be seen below in ***Figures 16*** and ***16***.

```
timer.setInterval(2000L, sendCounter);
```

Figure 16
New code for updating the counter value every 2 seconds

```
timer.setInterval(500L, sendCounter);
```

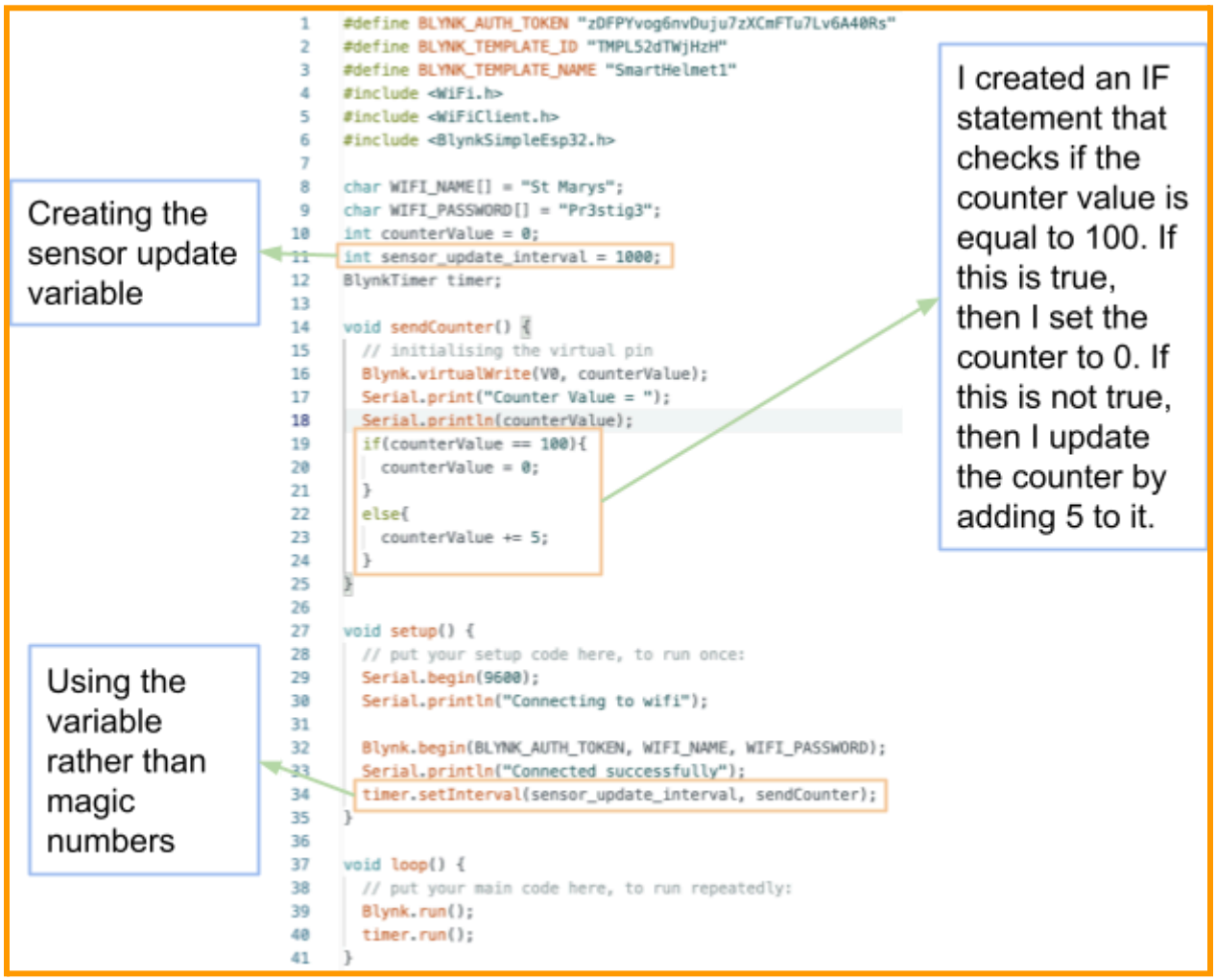
Figure 17
New code for updating the counter value every 0.5 seconds

However, as mentioned in ***Section 5.2 - Program Structure***, I want to minimise the use of magic numbers within my code. This is simply good coding practice as if I want to change these values later, I only need to update the main variable rather than find every use of the specific number within the code. As such, I created the **sensor_update_interval** variable which can be seen below in ***Figure 18***.

```
int sensor_update_interval = 1000;
```

Figure 18
Update interval variable

As a last challenge, I decided to change the code such that when the counter variable reaches 100, it resets. Otherwise, it will continue to increment by 5 each time. For this, I only needed to change the **sendCounter()** function, and could keep everything else the same (***Figure 19*** below).




```
#define BLYNK_AUTH_TOKEN "zDFPYvog6nvDuju7zXCmFTu7Lv6A40Rs"
#define BLYNK_TEMPLATE_ID "TMPL52dTWjHzH"
#define BLYNK_TEMPLATE_NAME "SmartHelmet1"
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

char WIFI_NAME[] = "St Marys";
char WIFI_PASSWORD[] = "Pr3stig3";
int counterValue = 0;
int sensor_update_interval = 1000;
BlynkTimer timer;

void sendCounter() {
  // initialising the virtual pin
  Blynk.virtualWrite(V0, counterValue);
  Serial.print("Counter Value = ");
  Serial.println(counterValue);
  if(counterValue == 100){
    counterValue = 0;
  }
  else{
    counterValue += 5;
  }
}

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  Serial.println("Connecting to wifi");

  Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_NAME, WIFI_PASSWORD);
  Serial.println("Connected successfully");
  timer.setInterval(sensor_update_interval, sendCounter);
}

void loop() {
  // put your main code here, to run repeatedly:
  Blynk.run();
  timer.run();
}
```

Figure 19
Code to reset the counter

7. Sensor Calibration

In order to ensure accuracy of my program, I need to calibrate the sensors and detect whether there are any errors within the components, such as offsets or any biases.

Types of Errors:

1. Offset error
    - An offset error is generally a flat absolute value that impacts every value. For example, if a thermometer had an offset of +2 degrees, when the room temperature is actually 25 degrees Celsius, the thermometer would measure 27 degrees. This is a systematic error that causes a zero error. However, if the offset is found, such an error can be accounted for relatively easily.
2. Bias
    - Bias refers to a sensor's systematic tendency to read slightly higher or lower over repeated measurements. An example of this would be if a weighing scale consistently gave a 3% error for anything being measured. A key difference between offsets and biases is that the former is input independent, while the later is input dependent. This means that the offset will be the same regardless of what is being measured. On the other hand, the absolute difference between the displayed value and the true value will vary depending on what is being measured if the sensor has a bias.
3. Noise
    - Noise refers to random errors. An example of this is if the actual value for a sensor reading is 50 but the displayed value keeps fluctuating at around 49.9. Noise could come from multiple signals in the surrounding area, or nearby electromagnetic radiation.

4. Drift
    - A drift error is one that results in the small change in values over time.

Static Calibration:

Before I start conducting proper simulation tests to see the values the sensor records during a range of activities, I need to calibrate the sensor using static calibration. This is a method where the sensor does not move. Below is the method I followed to carry out these tests:

1. Place the sensor on a breadboard to keep it flat and mark out where it is sitting on the table.
2. Place the breadboard flat facing upwards and connect the sensor to a power supply. Allow the sensor to warm up for 30 seconds and then collect the acceleration values in each of the X, Y, and Z planes over 60 seconds. Take an average of these values.

3. Repeat Step 2 four more times, giving five trials in total .
4. Take an average of all 5 trials.
5. Repeat Steps 1 - 4 when the breadboard is standing up on its short side, and then for when it is standing up on its long side (***Figure 20*** below).

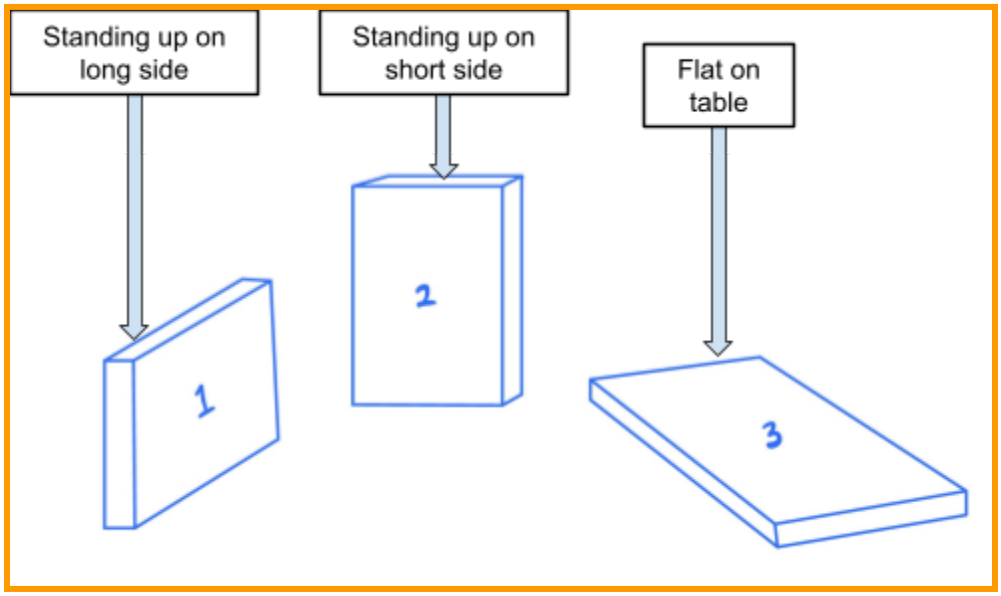


Figure 20
Drawing of sensor orientations for calibration testing

Table 2: *Flat on table:*

Test Run	Average X	Average Y	Average Z
1	0.31	0.06	10.64
2	0.31	0.06	10.65
3	0.31	0.05	10.69
4	0.32	0.06	10.70
5	0.31	0.06	10.69
Average	0.312	0.058	10.67

Table 3: *Upright on long side:*

Test Run	Average X	Average Y	Average Z
1	10.19	-0.02	0.31

2	10.19	-0.02	0.30
3	10.19	-0.02	0.31
4	10.19	-0.02	0.31
5	10.19	-0.02	0.31
Average	10.19	-0.02	0.308

Table 4: *Upright on short side:*

Test Run	Average X	Average Y	Average Z
1	0.28	9.76	-0.01
2	0.27	9.76	0.01
3	0.27	9.75	-0.12
4	0.27	9.76	0.01
5	0.27	9.76	0.01
Average	0.272	9.758	-0.02

The similarity of the trial suggests good repeatability. A more rigorous measurement of sensor noise would require analysing the variation or standard deviation of the individual stationary samples within each trial. The values are also what I expected them to be - the average acceleration in two of the planes should be $0ms^{-2}$, and in the third plane there should be an acceleration of around $10ms^{-2}$. From the data, we can also see that there is an offset in every plane, since for none of the tests can we see a perfect row of $0ms^{-2}$, $0ms^{-2}$ and $10ms^{-2}$. Despite this, these tests have shown that the sensor is quite accurate as the values are only offset by quite a small amount.

The static tests identified small systematic offsets and showed that the sensor gave repeatable readings. I did not subtract a fixed offset from every reading in the final code. This was because the crash-classification thresholds were developed experimentally using the same MPU6050, sensor settings and mounting arrangement. The final classifier also does not rely on one universal instantaneous acceleration threshold; instead it uses statistical features such as acceleration standard deviation, acceleration P99 and gyroscope P99 calculated from the experimental data. Therefore, the measured behaviour of this particular sensor is already represented within the thresholds developed for the prototype. A deployable product would still require a more rigorous multi-axis calibration and independent validation.

8. Testing

After successfully calibrating the sensor, the next step towards building the crash detection algorithm was to conduct real-world tests to gather data from the MPU 6050. Taking the static calibration tests previously carried out as Stage 1, I carried out 4 more stages of testing.

The question this research aims to answer is ‘how can data from the MPU 6050 be used to distinguish between normal motorcycle riding and genuine crashes while minimising false alarms?’. For the purposes of this initial prototype, I attached the sensor to a breadboard and a construction helmet (***Figure 21*** below). I decided not to use a motorcycle helmet just yet, as using a construction one is more cost effective for a prototype that will intentionally get damaged. To maximise the precision of these tests, I have increased the sampling rate of the sensor. This means that more data is recorded every second, allowing me to capture sudden spikes in values that could indicate a crash. When carrying out these tests, safety was the main priority, and as such, only controlled simulations (such as dropping the helmet) were used.

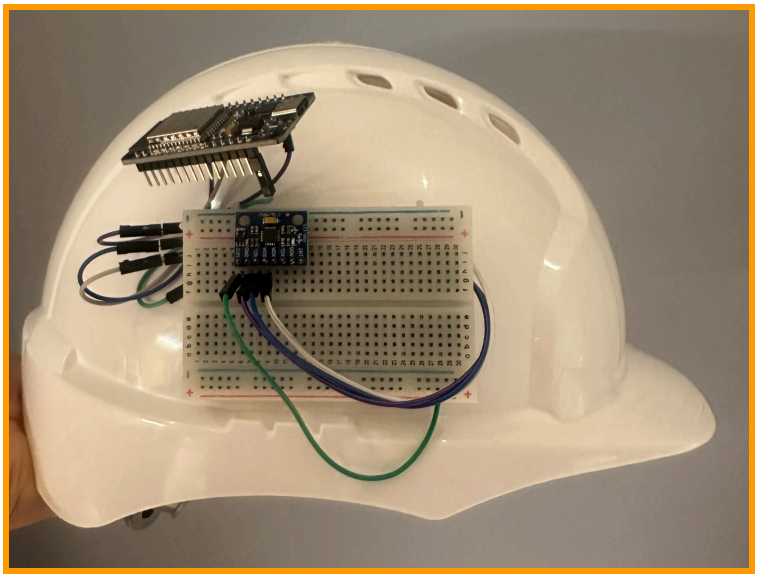


Figure 21
Sensor on helmet used for testing

Before I started the different tests, it was important to define the independent, dependent and control variables across all the stages.

The independent variables are the actions being carried out and the dependent variables are the acceleration and gyroscope values over time. The control variables are what I am keeping the same across each and every trial. These are the sample rate of the sensor, the code used to collect the data, the wiring of the sensor, and the orientation of the sensor. Fixing a breadboard to the helmet ensures that the sensor itself stays in the same position during every test.

Stage 2 - Everyday Motion:

Stage 2 involved measuring the accelerometer and gyroscope values during the following actions:

1. Walking with the helmet
2. Putting the helmet on
3. Removing the helmet
4. Placing the helmet on the table

Each of these tests were repeated 3 times.

Stage 3 - Riding Simulation:

Due to safety concerns and lack of access, I was not able to test the helmet on an actual motorcycle and instead carried out these tests in a car. Unfortunately, this was quite a significant limitation as I was not able to test the values on the sensor when turning left or right, as turning on a motorcycle is very different to doing so in a car. However I was still able to carry out the following tests, each of which were repeated 3 times:

1. Gentle forward movement
2. Faster forward movement
3. Gentle braking
4. Harder braking
5. Going over speed bumps

Stage 4 - Disturbance Tests:

The purpose of these tests was to gather sensor values for the events that are most likely to trigger false crash alarms. The idea is that if these specific motion patterns are detected, then the algorithm will be able to distinguish them as false alarms. I carried out each of the following tests 3 times through:

1. Shaking the helmet sideways
2. Shaking the helmet up and down
3. Tapping the helmet lightly
4. Tapping the helmet hard
5. Swinging the helmet
    a. To do this, I tied a rope to the helmet and swung it like a pendulum

Stage 5 - Controlled Impact Tests:

These tests were probably the hardest to carry out, and although they did not give me the exact values that would be expected in a real crash, the data given does demonstrate the pattern that a crash / sudden large impact would generate. I carried out the following tests 3 times each:

1. Soft impact
    - This involved throwing the helmet onto a mat with force
2. Dropping the helmet
    - This was done 3 times each from a short, medium and large height
3. Controlled crash into a wall
    - To carry out this test, I attached a rope to the helmet to create a pendulum. I then positioned the pendulum so that the pivot point is directly on the wall so that the helmet hits it at maximum speed.

9. Results

9.1 Results of Testing

To make the data easier to analyse, I created a feature summary table that lists the same statistics for each of the trials. This data along with the graphs that I created give crucial insight into the behavior of the sensor during different scenarios. Stage 1 testing is not included in these graphs, as it refers to the sensor calibration tests covered in **Section 7 - Sensor Calibration**.

Table 5: *Feature Summary Table*

Key:
Orange = Stage 2 tests
Blue = Stage 3 tests
Green = Stage 4 tests
Red = Stage 5 tests

Test	Mean Acc	SD Acc	P95 Acc	P99 Acc	Range Acc	Mean Gyroscope	SD Gyroscope	P95 Gyroscope	P99 Gyroscope	Range Gyroscope
Walking with helmet	10.453	1.448	13.248	15.969	12.255	0.808	0.258	1.751	2.121	2.300
Putting helmet on	10.319	0.368	10.893	11.743	5.369	0.237	0.144	0.580	0.716	0.859
Taking helmet off	10.417	1.738	11.255	12.472	63.178	0.262	0.271	0.862	1.402	2.451
Putting helmet on table	10.383	1.591	10.487	13.146	69.804	0.241	0.350	0.956	2.209	2.182
Slow forward 1	10.429	0.712	11.154	12.400	11.948	0.149	0.056	0.240	0.350	0.575
Slow forward 2	10.406	0.627	11.198	12.355	11.663	0.145	0.027	0.183	0.274	0.292
Slow forward 3	10.430	0.526	11.209	12.001	6.386	0.153	0.027	0.203	0.250	0.285
Fast forward 1	10.451	0.670	11.449	12.191	8.467	0.150	0.024	0.190	0.246	0.24
Fast forward 2	10.417	0.896	11.588	13.278	14.024	0.161	0.043	0.216	0.369	0.508
Fast forward 3	10.411	0.499	11.070	11.550	8.965	0.134	0.019	0.164	0.182	0.158
Soft break 1	10.431	0.257	10.826	11.231	5.079	0.141	0.011	0.166	0.172	0.083
Soft break 2	10.449	0.661	11.489	12.481	6.983	0.167	0.042	0.255	0.329	0.286
Soft break 3	10.433	0.317	10.928	11.436	3.730	0.140	0.008	0.153	0.170	0.092

Hard break 1	10.486	0.267	11.059	11.511	2.626	0.140	0.008	0.147	0.178	0.102
Hard break 2	10.417	0.371	11.009	11.463	5.727	0.139	0.012	0.160	0.167	0.115
Hard break 3	10.465	0.728	11.571	12.704	8.298	0.145	0.062	0.23	0.399	0.556
Shaking sideways 1	12.883	3.664	20.191	24.750	23.938	1.890	0.800	3.277	3.922	4.255
Shaking sideways 2	12.072	1.649	15.109	16.581	10.299	1.948	0.766	3.12	3.569	3.728
Shaking sideways 3	21.752	9.276	39.362	44.978	42.56	2.326	1.000	4.190	4.873	5.282
Shaking up and down 1	14.078	9.072	28.947	32.831	45.002	3.569	1.744	6.256	7.234	8.220
Shaking up and down 2	13.907	9.552	29.904	33.072	40.084	3.853	1.847	6.858	7.776	8.884
Shaking up and down 3	16.414	9.446	32.053	35.441	45.498	4.752	2.228	8.236	9.229	9.524
Tapping helmet lightly 1	10.678	2.898	12.344	27.940	37.484	0.177	0.102	0.390	0.731	0.951
Tapping helmet lightly 2	10.683	2.983	12.638	28.616	39.848	0.197	0.149	0.530	0.899	1.469
Tapping helmet lightly 3	10.715	3.027	12.291	29.539	35.851	0.209	0.166	0.602	0.930	1.352
Tapping helmet hard 1	10.952	4.128	13.362	31.543	82.301	0.182	0.232	0.249	0.786	10.184
Tapping helmet hard 2	11.744	7.620	15.792	54.936	102.276	0.254	0.365	0.801	1.745	6.832
Tapping helmet hard 3	11.557	6.808	14.215	52.154	81.892	0.227	0.264	0.687	1.570	2.600
Swing helmet 1	10.557	2.592	14.936	16.108	4.376	2.492	0.948	3.911	4.163	4.376
Swing helmet 2	10.200	2.971	14.618	15.489	4.459	2.238	0.897	3.789	4.462	4.459
Swing helmet 3	10.277	2.497	14.334	15.546	3.689	1.764	0.654	2.862	3.211	3.689
High drop 1	9.526	9.125	10.551	25.955	188.71	0.732	1.342	4.511	5.307	6.691
High drop 2	9.579	9.295	10.366	24.928	226.395	0.841	1.515	4.758	6.895	8.636
High drop 3	10.280	8.941	11.047	28.423	206.263	0.786	1.890	5.475	9.315	9.658
Medium drop 1	9.734	7.290	12.031	16.660	213.050	0.479	0.935	2.656	4.239	11.33
Medium drop 2	10.343	6.772	10.987	14.870	179.312	0.326	0.611	1.265	2.059	18.117

Medium drop 3	10.037	7.763	11.023	22.714	184.743	0.514	1.362	1.611	8.422	16.906
Low drop 1	10.400	5.704	10.531	14.659	170.245	0.222	0.416	0.712	1.447	17.533
Low drop 2	10.404	6.751	10.909	21.982	169.804	0.329	0.665	0.986	3.821	8.074
Low drop 3	10.400	5.665	10.541	13.875	156.770	0.239	0.453	0.579	2.279	5.607
Throw 1	9.702	8.705	11.251	28.602	218.251	0.504	1.113	2.232	5.925	16.310
Throw 2	9.981	7.949	11.577	17.086	221.537	0.854	2.127	4.241	12.346	14.301
Throw 3	9.947	8.175	11.314	26.630	222.559	0.626	1.512	3.061	7.134	14.592
Pendulum crash 1	9.758	3.986	11.395	15.203	20.487	0.811	1.321	2.967	6.860	20.487
Pendulum crash 2	9.743	3.920	10.766	15.041	20.487	0.730	1.439	2.998	7.107	35.563
Pendulum crash 3	10.024	4.969	11.203	18.366	20.487	0.912	1.593	2.896	7.274	40.146

From the numerical data, I also created four graphs for each trial: Acceleration vs Time, Gyroscope vs Time, Total Acceleration vs Time, Total Gyroscope vs Time. Since there are a total of 46 trials, I will only show the most important graphs below. The complete listing of all the graphs made can be found in the ***Appendix***. All of the graphs can be found in the supporting document attached.

9.2 Graphs

Putting helmet on table:

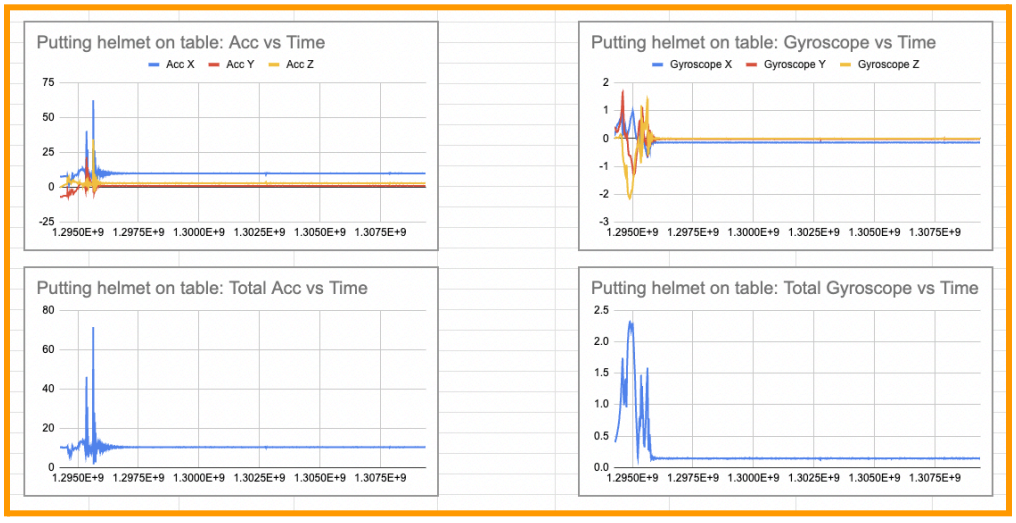


Figure 22
Graphs for putting helmet on table

As seen most clearly in Total Acc vs Time and Total Gyroscope vs Time in ***Figure 22*** above, at the moment of contact between the helmet and the table, there is a large spike in both values. This spike even reaches around 70m/s^2 for acceleration and 2.25rad/s. However, aside from this, we can see that there are hardly any fluctuations in any of the graphs.

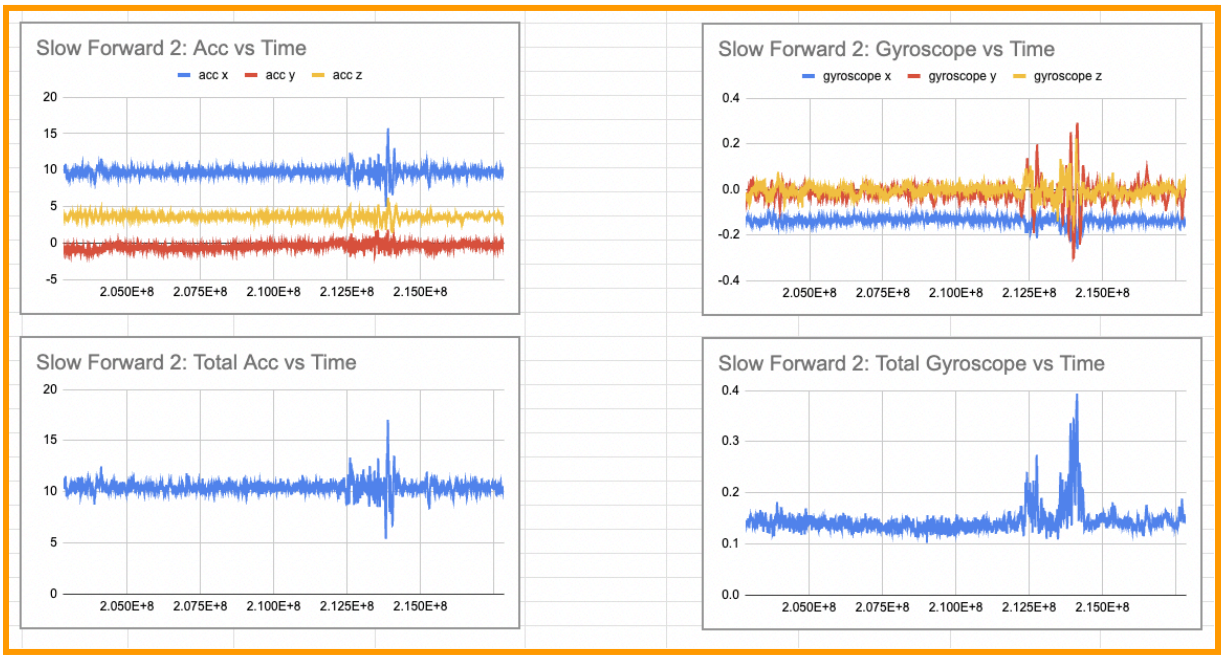


Figure 23
Graphs for moving forward slowly

From ***Figure 23*** above, we can see that this forward motion is not completely smooth, and that little fluctuations within the values should be expected. The spike visible in all four graphs could have been from a small bump on the road, which can still be interpreted as safe riding.

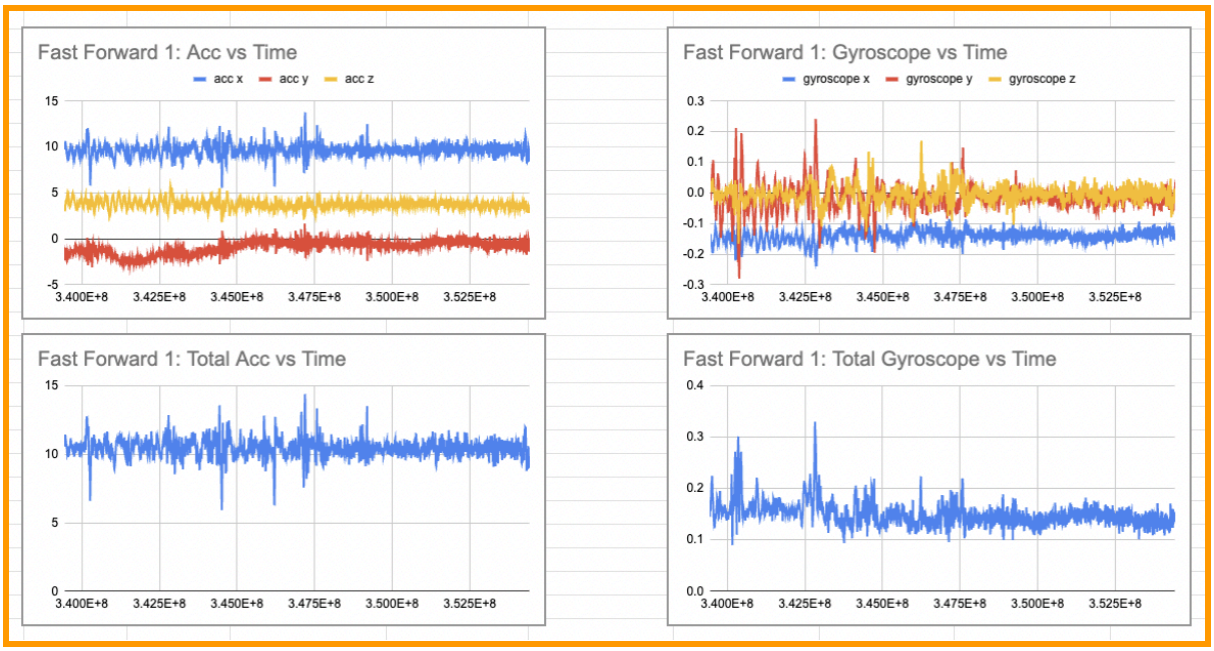


Figure 24
Graphs for moving forward fast

Figure 24 above follows the same pattern visible in the graphs for slow forward motion - little fluctuations about a persisting baseline. I was quite surprised to see that the values for acceleration in both ***Figures 24 and 23*** were very similar, with the former only having fluctuations of a greater magnitude. The range of gyroscope values for both motions were similar as well. These findings suggest that the main differentiator between slow and fast riding is the size of the fluctuations in sensor readings.

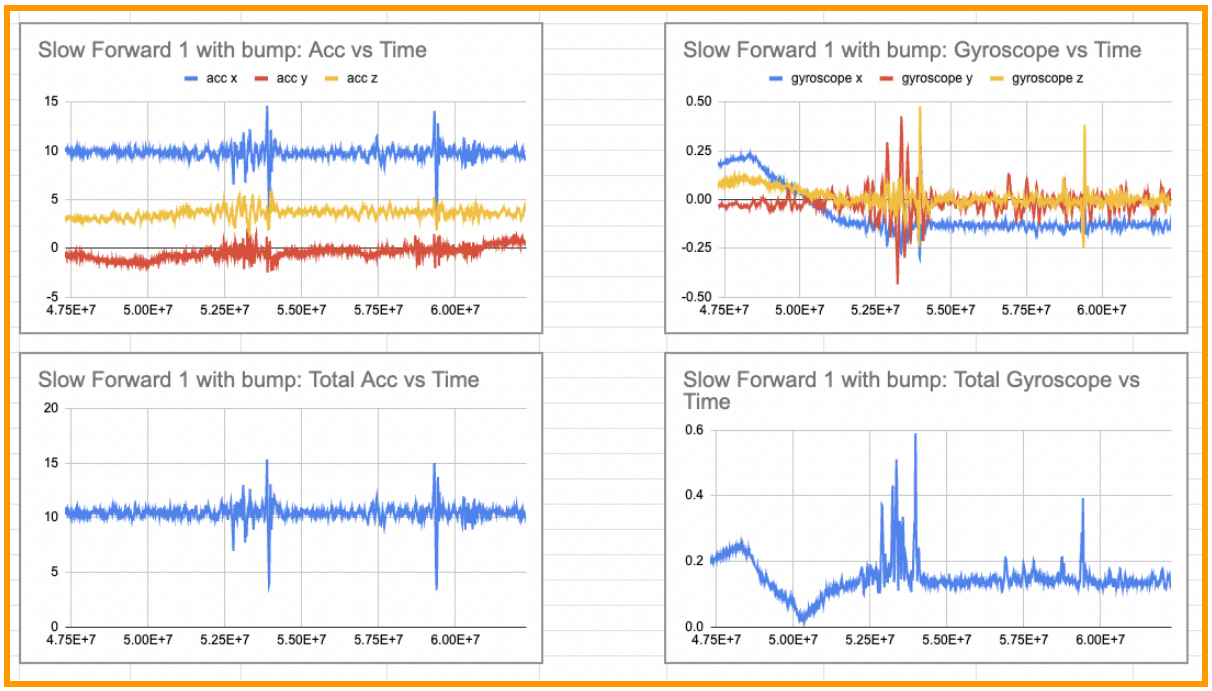


Figure 25
Graphs for moving forward slowly while going over a bump

When comparing ***Figure 25*** with the graphs for fast forward motion, we can see that the graphs for Acceleration vs Time and Total Acc vs Time do not change much, rather it is the gyroscope graphs that are impacted the most by going over a speed bump. From the graph showing Total Gyroscope vs Time, we can see that the values take a relatively significant dip when I go over the bump.

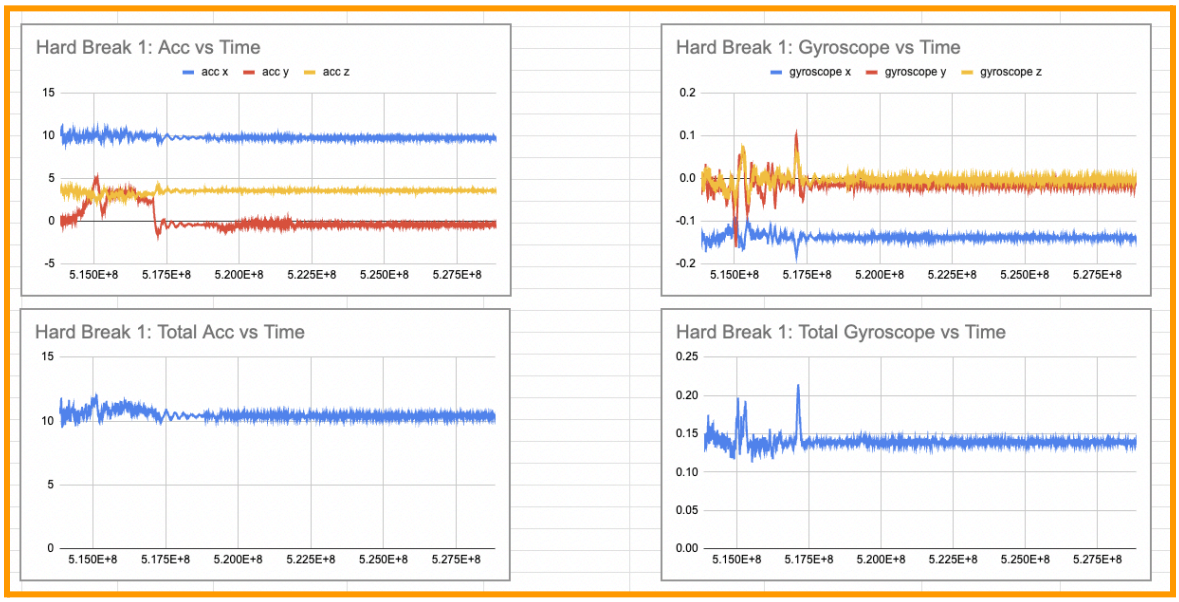


Figure 26
Graphs for hard braking

The graphs for hard braking (***Figure 26*** above) show that the sensor readings stayed fairly stable, with small fluctuations along the baseline. At the start, however, when the brake was applied, there was a spike in the gyroscope and acceleration, however, the magnitude of these was relatively small. In the Acc vs Time graph, there is a spike for the acceleration in the y-plane reading, however, occurs over a portion of time, suggesting that this event was more gradual as opposed to a sudden impact.

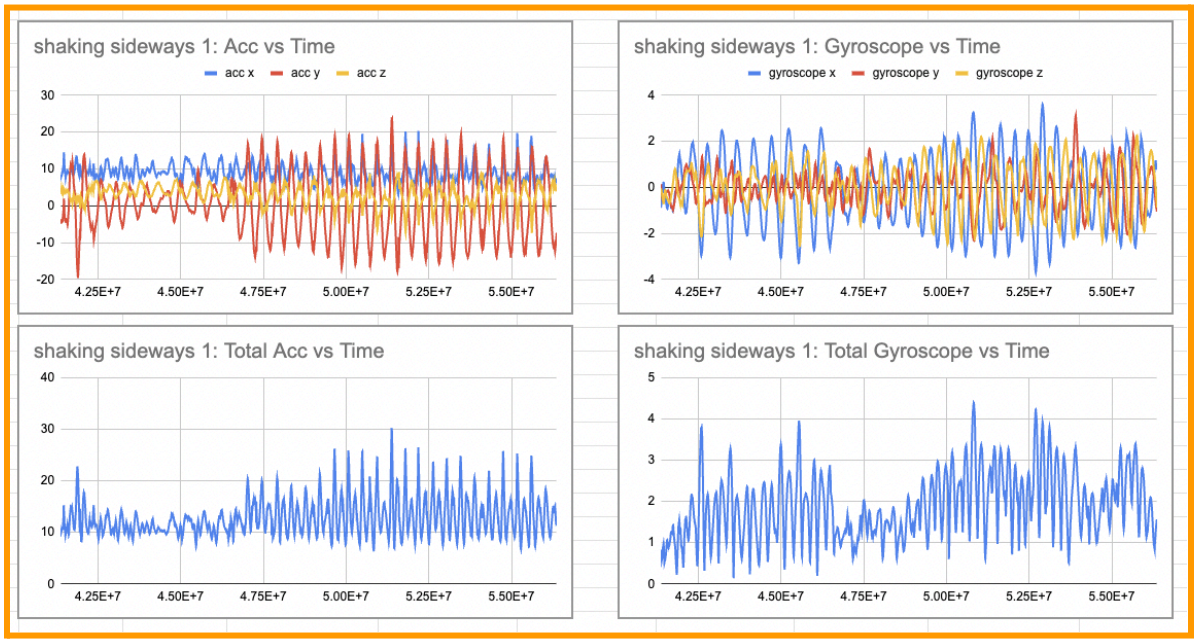


Figure 27
Graphs for shaking the helmet sideways

Shaking the helmet sideways (***Figure 27*** above) was one of the tests I carried out for false-positives. We can see from the graph that there are lots of spikes in the values of both acceleration and gyroscope. However, unlike an impact, these spikes repeatedly occur at fairly regular intervals, demonstrating the importance of taking into account the duration and nature of the helmet motion rather than just the raw acceleration or gyroscope values.

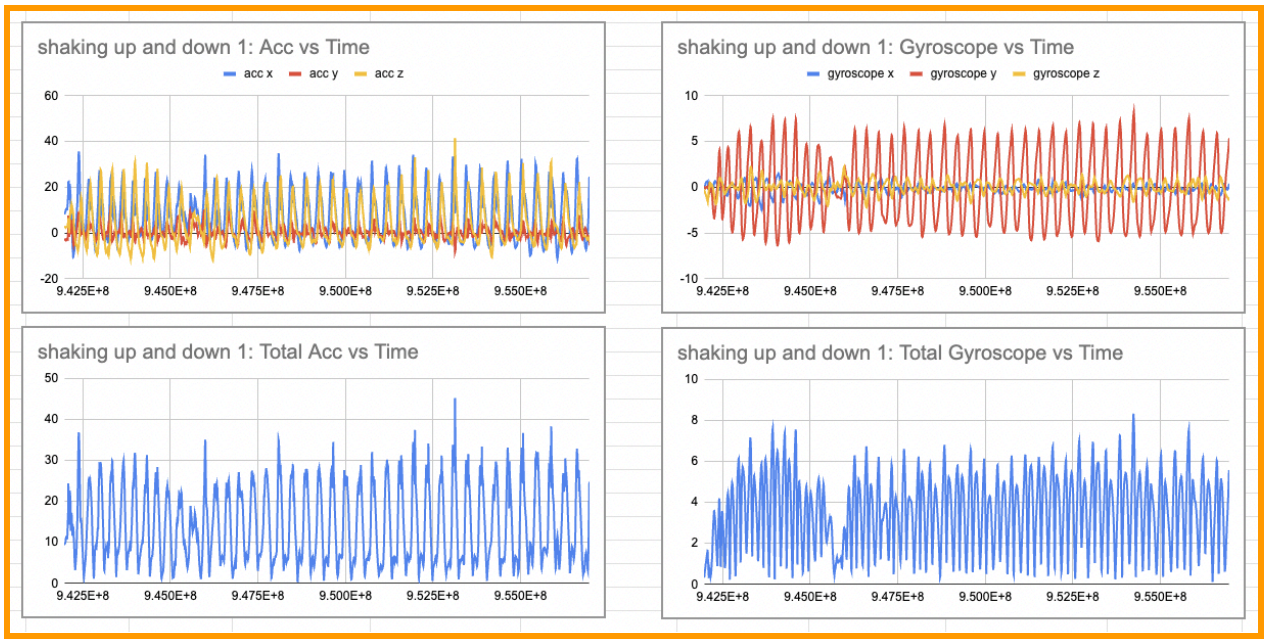


Figure 28
Graphs for shaking the helmet up and down

The idea of looking at the pattern of motion rather than just the values comes through even clearer in ***Figure 28*** above. For both acceleration and gyroscope, shaking the helmet up and down produced some of the highest values.

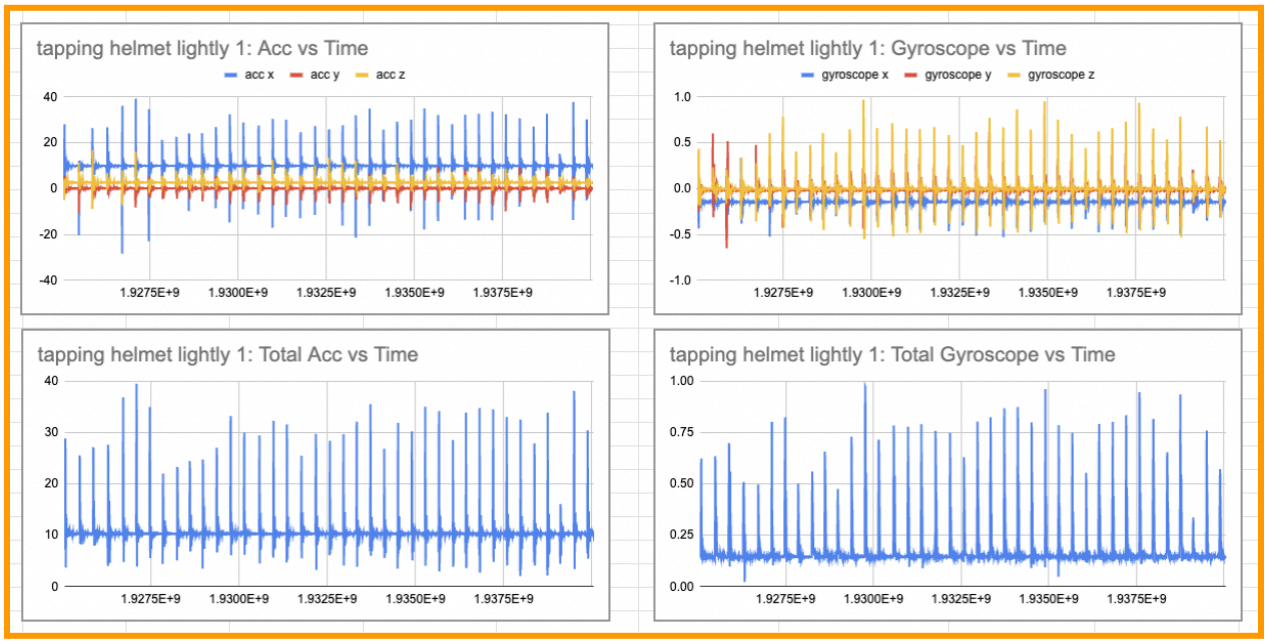


Figure 29
Graphs for tapping the helmet lightly

The graphs shown in ***Figure 29*** above follow a similar pattern to the graphs for shaking the helmet, in which they consist of a series of spikes evenly distributed. A key differentiator is that the spikes seen above are not as sinusoidal as the graphs in ***Figures 27 and 28***. For a crash, these sudden and narrow spikes would be seen, however, they would not be in the same repeated pattern as displayed in ***Figure 29***.

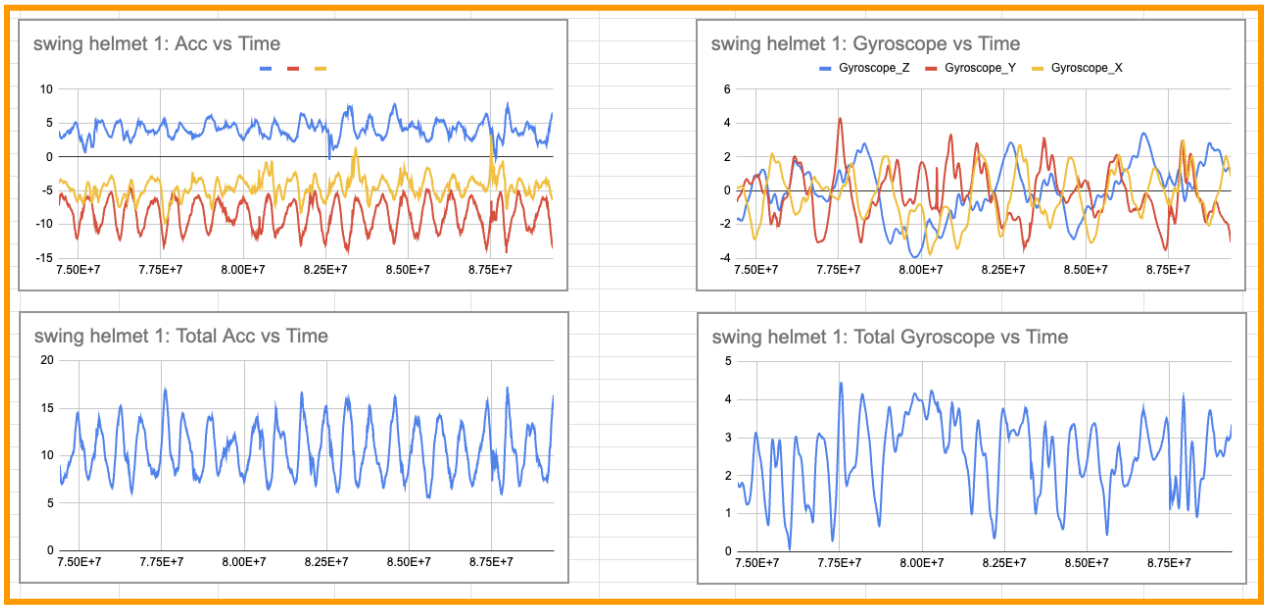


Figure 30
Graphs for swinging a helmet

The acceleration graphs in ***Figure 30*** above are the one of the most sinusoidal as seen so far. This is expected as I used a pendulum to swing the helmet. The values of acceleration and from the gyroscope are not too dissimilar from those during normal riding motion. These graphs give swinging the helmet a distinct pattern that differs to that of an impact.

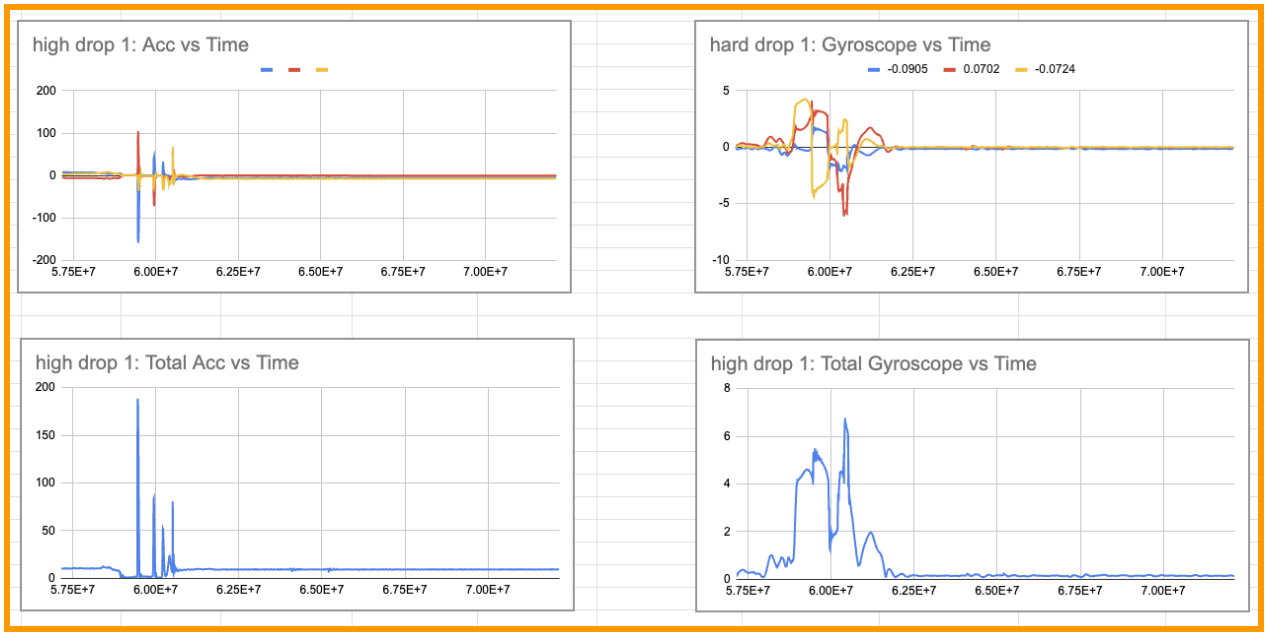


Figure 31
Graphs for dropping the helmet from a large height

All four of the graphs pictured in ***Figure 31*** above show one main spike followed by stable readings. This spike happens when the helmet crashes into the ground, and looking at the graphs for a drop from a medium and low height, the actual values reached by the sensors do not change much. This suggests that perhaps the orientation of the helmet impacts the readings as well. Therefore, rather than simply taking the maximum acceleration or gyroscope, it is important to look at the surrounding contextual motion of the helmet.

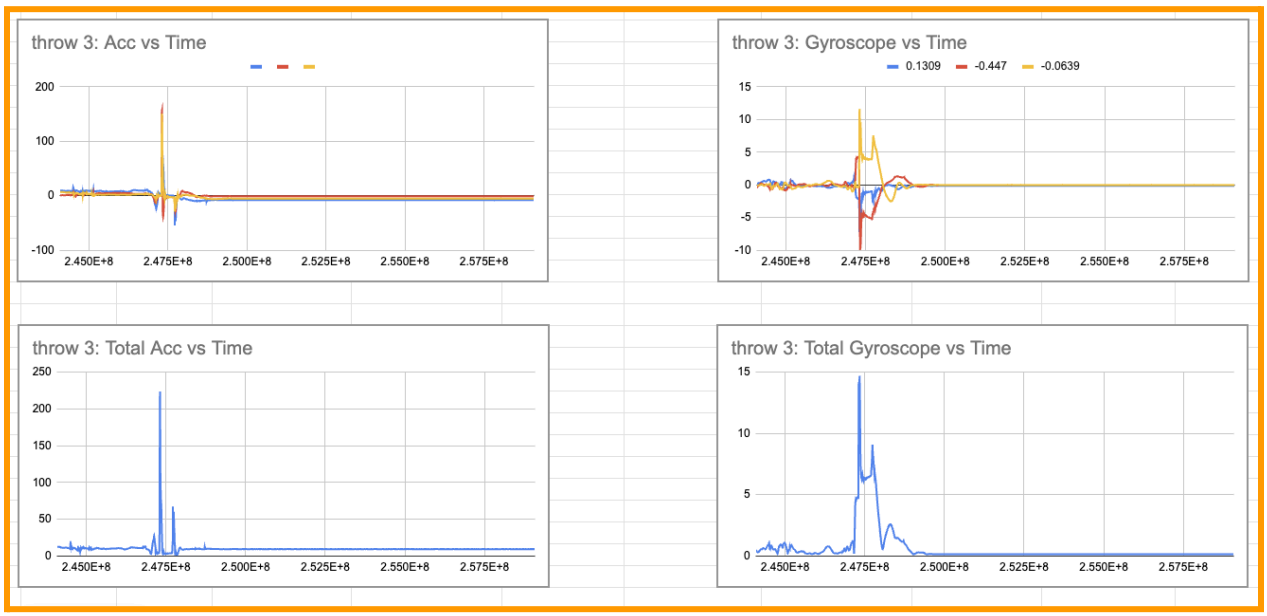


Figure 32
Graphs for throwing the helmet from a set height

The overall shape of the graphs in ***Figure 32*** is very similar to the shape of the graphs for simply dropping the helmet (with no external force applied) from different heights. The values reached, however, by the accelerometer and gyroscope do increase significantly. This suggests that the sensor data from a crash would form a trace that has a sharp sudden spike of large magnitude.

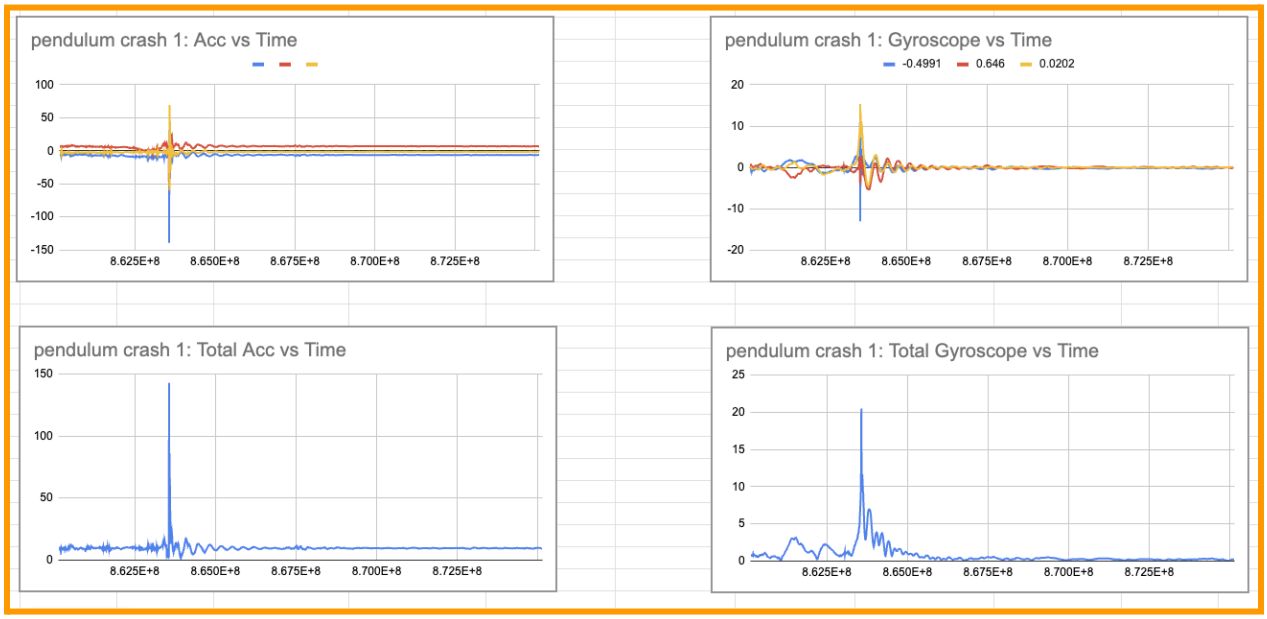


Figure 33
Graphs to show using a pendulum to crash the helmet

Once more, the same pattern of a sudden narrow spike is present. The data surrounding this spike is also quite stable with a few fluctuations. The accelerometer and gyroscope values measured by the MPU 6050, however, are lower than those for dropping the helmet from a large height. This could be a result of not using a long enough pendulum or the helmet hitting the wall at a different angle.

10. Feature Selection

10.1 Selecting Parameters

As seen from the feature summary table in **Section 9 - Results**, for each of the trials, I calculated multiple summary statistics for both acceleration and gyroscope. This was done so that I could select the features that provided the most insight into the nature of the motion of the helmet. From my analysis, I concluded that the standard deviation and 99th percentile values varied the most depending on the type of motion being carried out. This meant that they could essentially be used to differentiate between different states such as normal riding and a disturbance.

10.2 Standard Deviation

Standard deviation is a statistical measure of how much data varies around its average, and is given by the formula (***Figure 34*** below):

$$SD = \sqrt{\frac{\Sigma(x-\bar{x})^2}{n-1}}$$

Figure 34
Standard deviation formula

A helmet moving with little disturbance will generally have a low variation, while sudden impacts will result in a larger variation. The standard deviation of the acceleration was more useful than the mean, as the latter stayed close to $9.8ms^{-2}$ for many tests, including those in Stage 4. This was because the majority of the values of acceleration stayed close to the gravitational field strength of the Earth, as the impact event only lasted for an extremely short amount of time.

10.3 99th Percentile

The 99th percentile is the value below which 99% of the data gathered lie. Unlike the maximum value, this is not dominated by one unusual reading. This characteristic is extremely important, as looking at the graphs for placing the helmet on the table, there is one significant spike in acceleration values when the helmet hits the table. This value can also be used to distinguish different patterns. For example, repeated shaking of the helmet will produce a high 99th percentile, whereas a sudden impact such as a drop will result in a low 99th percentile.

Only measuring data for acceleration would also not give a complete picture of the current event. As such, I also decided to use the 99th percentile of the gyroscope to give insight into if there has been any unusual rotational movement as well as linear acceleration.

10.4 Alternative Parameters

Although the other values calculated during testing (mean, maximum, minimum, range, P95, individual X/Y/Z values) were useful when analysing my results, they did not add sufficient value to the crash detection algorithm to justify the extra complexity and computational load.

- Mean: similar across many different tests
- Maximum: impacted by a single isolated spike
- Minimum: not useful in differentiating different event types
- Range: impacted by anomalous readings which could misleadingly classify normal movement as dangerous
- P95: gave weaker distinction between the classes than P99
- Individual X/Y/Z: values in each plane impacted by the orientation of the sensor

11. Writing the Crash Detection Algorithm

11.1 Overview

The full commented version of the code can be found in the **Appendix**, which is a separate file**.** As discussed previously in the documentation, the helmet uses the MPU 6050 sensor as an accelerometer and as a gyroscope. The sensor takes 200 samples per second, which is fast enough to detect sudden changes in motion. This is crucial as otherwise, short impacts could be missed. Instead of only using the current acceleration value, the program holds a rolling window of 3000 of the most recent sensor readings and calculates the standard deviation and 99th percentile of total acceleration, as well as the 99th percentile of the total gyroscope. These features were decided by looking at the data within the feature summary table in **Section 9 - Results**. For an impact like event, the acceleration and gyroscope graphs have one spike over a very short amount of time. Standard deviation was chosen because it measures the spread of data over an analysis window. Strong impacts or repeated motion increase this spread of data.

The algorithm categorises the events into three groups - Normal, Disturbance and Impact-like. Normal motion includes walking with the helmet, or ordinary helmet handling. Disturbances include many of the motions from the Stage 4 testing, such as shaking or tapping the helmet. This is because such motions may give high sensor readings, but are not necessarily crashes. Finally, an impact-like event is only detected if the combination of the acceleration and gyroscope patterns match the patterns observed during the drop, throw and pendulum tests as these were the closest conditions I could get to simulating a real crash. I chose this multi-factor classification rather than one threshold value as it would be more reliable.

If the algorithm detects multiple impact-like events, an emergency alert is not immediately sent. Rather, the 5 LEDs on the helmet start flashing and the rider is given 10 seconds to cancel the emergency by pressing the push button if they are safe. If this does not happen, then the ESP32 uses the GPS module to obtain the latitude and longitude coordinates of the rider and sends an emergency event through Blynk with this information. The program also generates a Google Maps link to the location of the GPS fix, improving the efficiency of the alert.

As mentioned previously in the documentation, I used decomposition to help me write the algorithm. This means that the overall program is split into individual functions with clear roles. For example, there are functions to handle the sensor readings, calculations of the statistical features for acceleration and gyroscope, as well as reading the GPS data. This made the program easier to test as well as made it possible to change one function without needing to rewrite the whole code.

It is important to note that since the threshold values used in my program were selected from the results of the testing carried out, the product created is therefore a prototype rather than a fully-certified real-world motorcycle crash detection system.

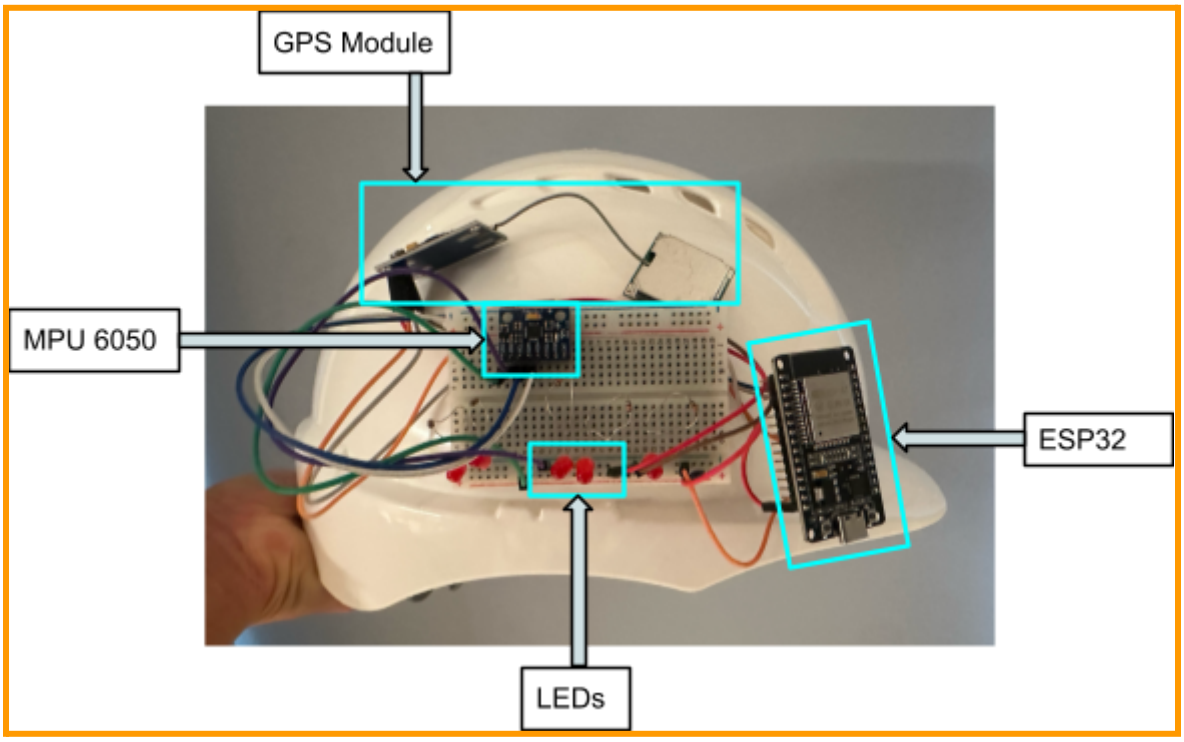


Figure 35
Labelled image showing helmet with all the components

11.2 Rolling Window

A 15-second rolling window is used in the algorithm to store the most recent 3000 data points. Since the MPU 6050 operates at a frequency of 200Hz, this means a reading is taken every 5ms. This collection of data was needed as using a single data point would be unreflective of the overall motion of the helmet.

I chose this specific size of the rolling window, as for each of the tests carried out, I collected 3000 values as well. Keeping the same sample size would help improve the reliability of comparing the statistical features of the live data and of the reference data.

When a new sample is collected, the oldest reading in the rolling window is removed and replaced. This ensures that the window is being constantly updated with the most recent sensor readings, helping reduce the latency of the algorithm.

11.3 Algorithm Classification Conditions

The classification of motion types happens in the **classifyEvent()** function. The first check determines whether there has been an impact-like event or not. For this to happen, the data has to follow either of these two sets of conditions:

1. Impact A (linear acceleration based):
    a. SD Acceleration >= 5.5
    b. P99 Acceleration < 30.0
2. Impact B (rotation acceleration based):
    a. SD Acceleration >= 3.5
    b. P99 Acceleration < 20.0
    c. P99 Gyro >= 6.0

If none of these conditions are met, then the algorithm checks if the motion matches a disturbance. For this to be true, the data has to meet either of the following criteria:

1. Acceleration Criteria:
    a. SD Acceleration >= 2.5
    b. P99 Acceleration >= 25.0
2. Gyroscope Criteria:
    a. P99 Gyroscope >= 3.0

If none of these conditions are satisfied, then the event remains under the label of ‘NORMAL_EVENT’. Thus, the classification function can be summarised with the following flow chart (***Figure 36*** below).

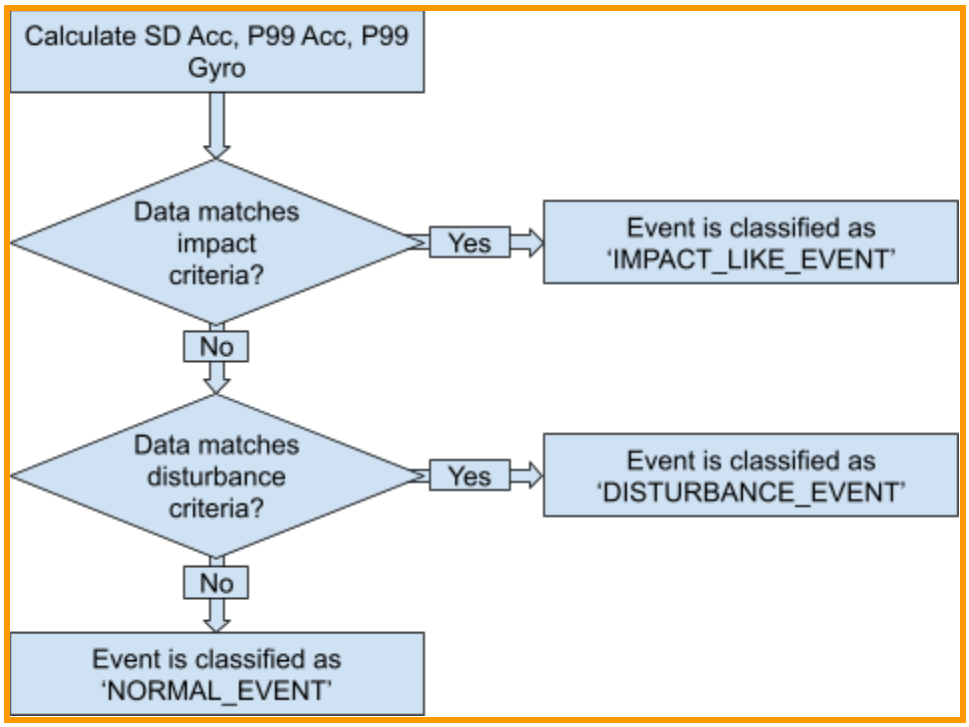


Figure 36
Classification function flow chart

11.4 Updating the Blynk Dashboard

From the dashboard featured in **Section 6 - Blynk Development**, I have added multiple features to accommodate the progress with the crash detection algorithm. The dashboard, which can be seen below in ***Figure 37***, serves as a visual interface for the user to easily interpret all the information being produced by the algorithm.

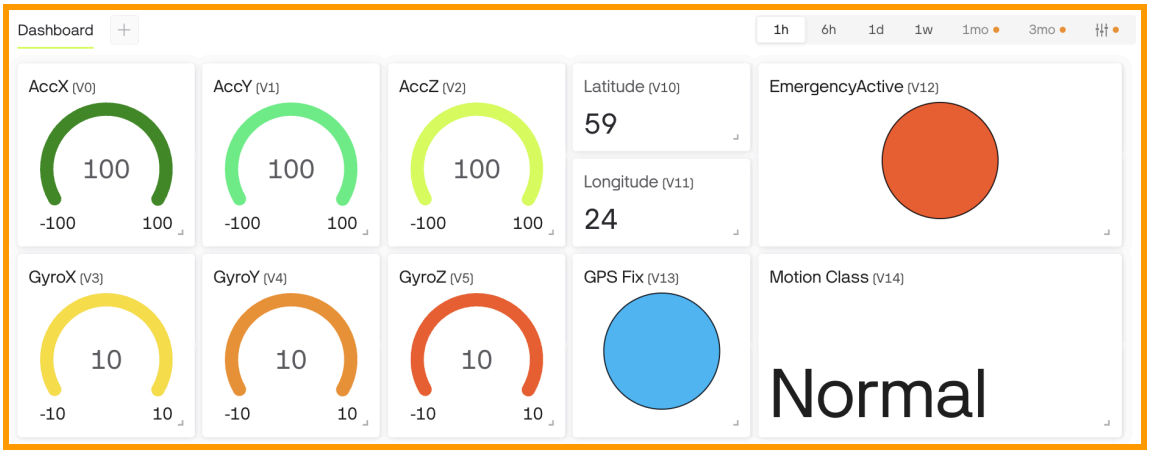


Figure 37
New Blynk Dashboard

I kept the same gauges from the old display as I found this was an effective way to communicate the acceleration and gyroscope values. For the latitude and longitude, I decided to stick with standard labels, as I wanted to prioritise clarity for such an important feature. Both the GPS Fix and EmergencyActive buttons start off with a white fill, and then change colour depending on if a fix has been secured or if an emergency alert is being sent respectively. Keeping these displays as completely visual helps mitigate complexity. Finally, I added a text label for the motion class, which displays Normal, Disturbance or Impact-Like depending on the algorithm.

11.5 Final System Validation

Table 6: *Final System Validation Results*

Test	Expected result	Expected Alert behaviour	Number of fresh trials	Correct classifications
Walking/normal movement	NORMAL	No alert	5	5
Shaking	DISTURBANCE	No alert	5	5
Hard tapping	DISTURBANCE	No alert	5	5
Controlled impact/drop	IMPACT-LIKE	10 s rider check	5	5
Cancel button	Cancellation	Alert stopped	3	3
No cancellation	Emergency event	Blynk notification received	3	3
GPS test	Valid location	Maps link opened correctly	5	3

As can be seen in ***Table 6*** above, the helmet passed all the tests apart from the GPS trials. Unfortunately, the module was a bit unreliable in securing a GPS fix, however, once a location was confirmed, it was extremely accurate. It is important to remember that the validation set was small and did not contain real motorcycle crashes, so this should not be interpreted as 100% real-world crash-detection accuracy.

I also recorded a demonstration video of the completed prototype showing the system operating as a whole, including live motion classification, the warning LEDs, rider-cancellation function,

Blynk interface, emergency notification and GPS/location functionality. This video has been submitted as supporting evidence alongside the report.

12. Problems Encountered

Throughout the development of this project, there were many problems faced, especially during the coding stage. These challenges taught me how to adapt and think resourcefully to make the most of the materials I already have.

The first major issue I faced was figuring out how I would store all 3000 values from the sensor during testing. I needed some way to access all the data after the trial was done so that I could process it. Printing the values to the serial monitor and copying all of them over to my spreadsheet was extremely inefficient and took far too long, since for every sample, the following values were being printed:

- Trial number
- Time gap
- Time since start
- Acceleration in the x-plane
- Acceleration in the y-plane
- Acceleration in the z-plane
- Gyroscope in the x-plane
- Gyroscope in the y-plane
- Gyroscope in the z-plane
- Total acceleration
- Total gyroscope

This prompted me to try and find a better way to store all of the data. When doing my CS A-Level coursework, I wrote all the values generated by the calculations being carried out by the program to a CSV file, giving me the idea of doing the same here. Since I had not done something like this before with Arduino, I asked ChatGPT to write me a sample code that would store all of the data being transmitted through the serial port the ESP32 was connected to and store it in a file. The result was a Python file (***Figure 38*** below) that successfully produced .txt files of all the data. This sped up testing quite a lot, helping save time in processing the data.

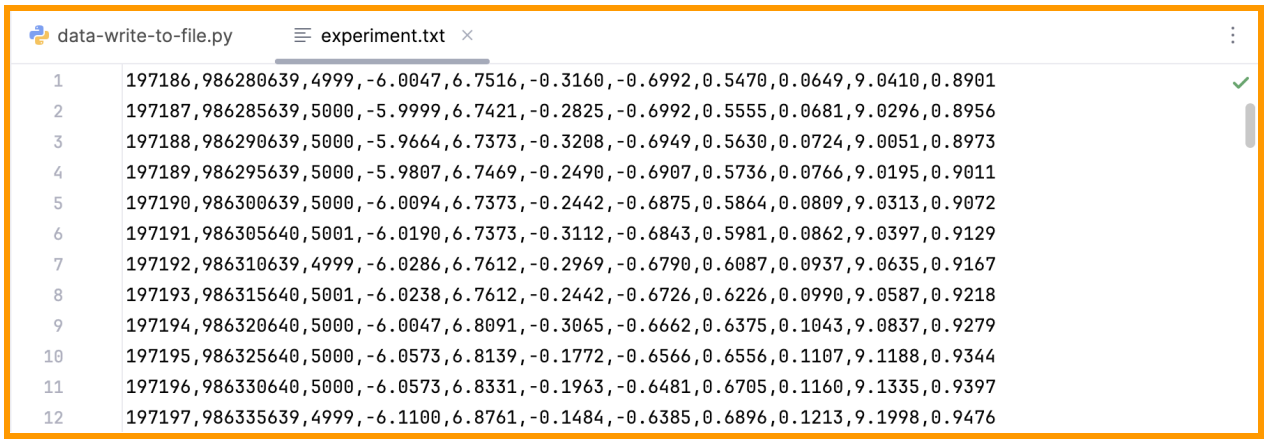


Figure 38
Screenshot of an example .txt file produced by the Python program

When I first ran the full crash detection algorithm on the ESP32, nothing was being output to the serial monitor and instead, the Arudino IDE continuously crashed. This was quite concerning as I thought that the code was too computationally heavy and so would not run on my laptop. As a first attempt to diagnose the problem, I removed the Blynk connection from the code, which did not fix anything. This told me that the problem was with the main algorithm itself rather than with Blynk. I then tried changing the baud rate within the code and the serial monitor, as this had fixed some issues in the past. Fortunately, this fixed the issue since I realised that I had accidentally changed the GPS baud rate. The ESP32 UART baud rate must match the GPS module's configured baud rate; my module was operated at 9600 baud.

Since this project was the first time I was using Blynk, there were a number of problems that I faced during the process, which only strengthened my understanding of the platform. For example, when I was testing the algorithm after integrating it within the helmet, I was initially receiving emergency alerts on my phone after impact-like events. However, when I tried to test it again, I was no longer receiving anything. Going through the code, I could not find any issues, or anything that had changed from the previous test. I then decided to find the problem on the Blynk console itself. Going through the settings of the Blynk event I had created to send these emergency alerts, I found the ‘Limits’ section (***Figure 39*** below). Here, I realised that, by default, the settings restricted notifications to once per hour, which is why I only received the initial alert and none of the subsequent ones. To fix this, I simply changed the cool-down time to 1 minute. Upon retesting, every impact-like event which was not cancelled successfully sent a notification to my phone.

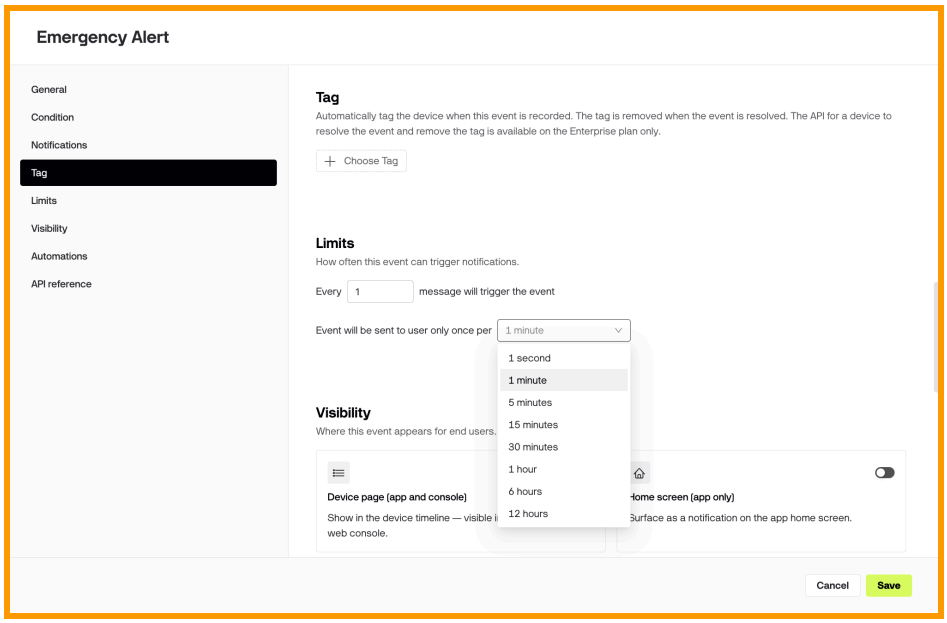


Figure 39
Screenshot of the Blynk Event Limits Settings

Another main issue was with the GPS module. When I first connected it to the ESP32 and ran the algorithm, I was unable to get a GPS fix even after waiting for quite a few minutes. The first step I took was to check that the module was receiving power using a multimeter. As expected, I was measuring a value of 3.3 volts, meaning that power was not the problem. From my previous issue with the Arduino IDE crashing, I knew that the baud rate of the GPS should not be changed. As such, I tried moving the helmet to different locations, and after I eventually took the GPS module outside, I finally started to get GPS fixes (***Figure 40*** below). The solution was to simply take the module outside so that it could secure its first fix, since inside a house, even though I was right next to a window, radio waves could still get absorbed and reflected by walls and surrounding structures.

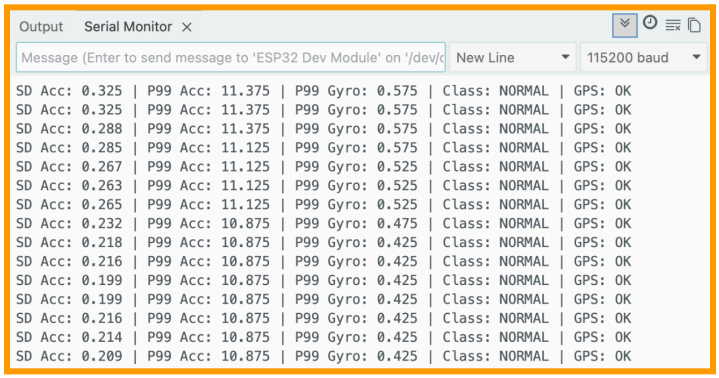


Figure 40
Image showing serial monitor once GPS fix secured

13. Limitations

As explained previously, the product that I made works very well as a prototype, but does still have a few limitations. The main one is the lack of a motorcycle during testing, as I was unable to carry out certain motions such as turning, as well as the fact that the data I gathered may not be extremely accurate since the mechanics of a car and motorbike are different.

The MPU6050 was appropriate for a low-cost prototype, but its ±16 g accelerometer range may saturate during severe real motorcycle impacts. A road-ready version should therefore investigate a dedicated high-g accelerometer alongside the gyroscope. It would also require formal calibration, temperature testing and independent validation.

The prototype also relies on Wi-Fi for Blynk communication. In the current firmware, the Blynk connection occurs during startup, so poor connectivity can delay the system becoming fully operational. A future version should separate local crash detection from cloud connectivity so that sensing, LEDs and the rider-confirmation button continue operating even without internet access.

The final algorithm is also in its initial stage, and so can be developed further to include a more complicated multi-factor classification function.

14. Wider Implications

The wider purpose of this project is to improve motorcycle safety by reducing the time it takes for help to be requested after a crash. In the prototype, confirmed impact-like events can automatically generate a Blynk emergency notification containing the rider's location and send it to a designated recipient. During testing, I deliberately did not connect this system directly to 999, to avoid accidental calls. However, the event and recipient structure could be expanded in a deployable version to include an authorised emergency-service communication route, subject to the necessary technical integration and approval. This will be especially effective in rural areas or at night, where there may not be many people around to call for help. As seen in ***Figure 41*** below, 70% of fatalities caused by motorcycle accidents from 2021 to 2025 happened on roads in rural areas[13]. This is one situation in which a reliable automated crash-alert system could potentially reduce the delay before assistance is requested.

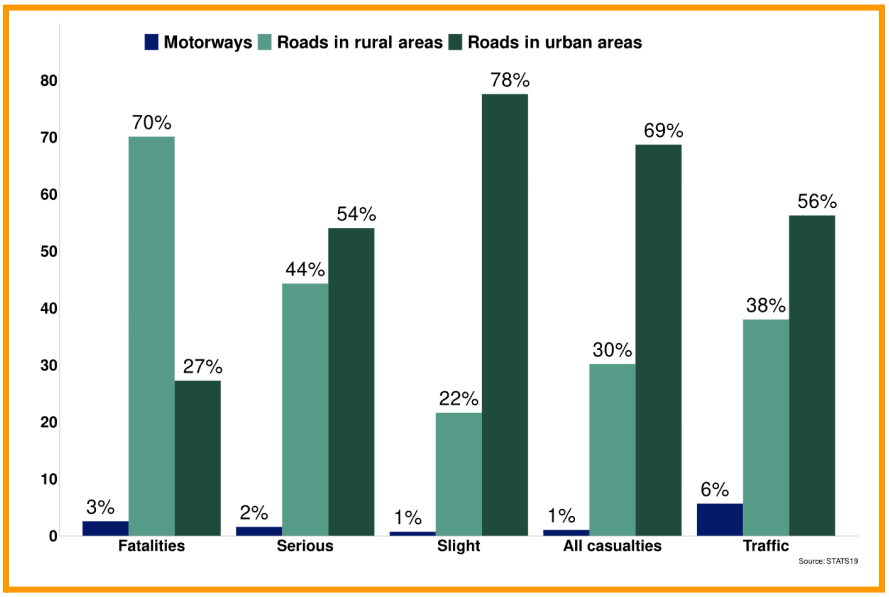


Figure 41
Graph showing percent of motorcycle accidents on different types of roads[13]

Furthermore, through reducing the number of false alarms sent to emergency services, my product helps prevent emergency services becoming unnecessarily strained - taking away resources and staff from critical matters. However, since this project is very much a first prototype, it will need continuous development to become market-ready. At the core of these future iterations, will be the idea of affordability. For riders buying a brand new motorcycle helmet, having the crash detection system embedded within the product works well, however, for those who already have a helmet, this approach would not be sustainable. Instead, having the option to retrofit the system onto an existing helmet will not only increase the appeal of the

product, but will also be better for affordability, as just buying the safety device would be far cheaper than buying a whole new helmet.

15. Evaluation

Overall, I am very pleased with how this project turned out. The end product effectively achieves all of the objectives I had set out at the start of the project, and the process has helped develop my skills as an engineer. The challenges covered in **Section 12 - Problems Encountered** have pushed me to look past the obvious answers and have advanced my critical thinking abilities.

My Gantt Chart worked really well in keeping me on track during the project, and I ended up finishing everything in the week of October 5th, as planned, although some individual tasks shifted within the schedule. Scheduling projects task by task is a skill which will be incredibly helpful as I start more complex projects at university. Looking back, planning the project on a week by week basis was particularly useful as it gave me buffer time within the weeks themselves if I needed to spend more hours on a certain task. One example where this came into use was during the helmet assembly, as I had run out of wires mid-way and needed to wait a couple of days for new ones to arrive.

15.1 Skills Gained

This project introduced me to multiple new techniques and was the first time I actually carried out a full sensor calibration routine. My mentor guided me through the process of testing for offset and noise within the sensor values, a skill which is necessary in ensuring the accuracy of my future projects. Through setting up the different stages of testing, I also learnt a lot about experimental design and procedure, especially when carrying out the risk assessment.

The final algorithm is incredibly lengthy, and so going through my work trying to find the cause of different issues was undoubtedly a challenge but it developed my debugging skills and, as a more mechanically minded person, this helped grow my confidence with coding.

Although I knew about it, this project was also the first time I actually worked with the Internet of Things. I really enjoyed using Blynk and found it to be an incredibly interesting platform. As a result of this project, I even used Blynk in my EPQ project to control the robot dog that I made. I definitely see myself incorporating this technology in my future work, and am excited to see how it can help take my projects to the next level.

15.2 What I Would do Better

One practical aspect I would improve on if I were to do this project again, would be to ensure that I have all the necessary materials as well as spare components before I start building everything. On multiple occasions, my components broke or I ran out of them, slowing down my progress. I would also keep a more detailed journal of my progress every time I worked on the project, as this would help me when critically evaluating the work I have done. I also try to make

the helmet battery powered, as this would have been very helpful during the testing stages where I had to wear the helmet.

16. Conclusion

The overall aim of this project was to create a smart motorcycle helmet that successfully detects a crash and sends an alert to emergency services. For the purposes of this prototype, I programmed the notifications so that they were sent to my phone rather than 999. Overall, the project successfully measures acceleration and gyroscopic values and accomplishes this aim at the prototype level, but it will need further development for it to be ready for real-world use.

16.1 Comparison against project objectives

Table 7: *Testing the Project Against its Objectives*

Objective Number	Objective Description	Status	Evidence
1.1	Constantly collect accelerometer and gyroscope data from the ride	**PASS**	- The rolling window in the crash detection algorithm constantly stores the 3000 most recent acceleration and gyroscopic values
1.2	Process the data and calculate statistical values for the samples	**PASS**	- The standard deviation and 99th percentile of the acceleration values are calculated - The 99th percentile of the gyroscope values is calculated
2.1	Calibrate the sensor so that the data I collect is accurate and representative of the current situation	**PASS**	- I carried out static calibration tests in three orientations and five repetitions to quantify the sensor's offsets and repeatability. - I did not apply a fixed numerical offset correction in the final firmware because the classification thresholds were developed from data collected using the same sensor and

			configuration, and the algorithm uses SD and P99 features rather than a universal instantaneous acceleration threshold. - Formal calibration would still be required for a deployable product.
3.1	Distinguish normal riding from crash-like events using accelerometer and gyroscope data	**PASS**	- The crash detection algorithm successfully distinguishes between normal riding, disturbance events and an impact-like occurrence. - I tested the reliability of the algorithm by carrying out the same motions for Stage 4 and 5 testing, all of which were correctly grouped under 'DISTURBANCE' and 'IMPACT-LIKE' respectively. - I also wore the helmet and replicated the motions in Stages 2 and 3, all of which were once again correctly identified as 'NORMAL' motion events.
3.2	Identify scenarios which could create false crash alarms and create measurements to reduce nuisance alerts	**PASS**	- Stage 4 testing and the creation of the 'DISTURBANCE' motion class effectively handle these false-alarm scenarios and they do not trigger an emergency alert

3.3	Develop a multi-stage crash detection algorithm that relies on multiple factors	**PASS**	- The crash detection algorithm uses multiple different conditions to classify events as 'IMPACT-LIKE' or 'DISTURBANCE' - For example, depending on the type of crash, different statistical features have to meet different criteria.
3.4	Trigger an automated emergency alert and location message using the Internet of Things, with provision for future authorised emergency-service integration.	**PASS**	- Whenever the algorithm detects two or more consecutive 'IMPACT-LIKE' events and the user has not cancelled the emergency, I receive an email as well as a notification from Blynk alerting me of the accident as well as the location of it in a Google Maps link.
4.1	Provide the user with a method of cancelling the emergency alert if they are safe	**PASS**	- The system includes a push button on the side of the helmet, which the user can push if they need to cancel the emergency alert

16.2 Technical Findings

From the tests I carried out, as well as from analysing the data collected, I was able to conclude that crash detection cannot reliably be built on the basis of a singular threshold. My experimental dataset showed that a single threshold was insufficient for this prototype. Seeing the patterns in the different statistical features across the various movements carried out, made me realise just how difficult it is to find a singular value that separates safe riding from an accident. Considering multiple aspects together help produce a clearer and more detailed image of the current state of the rider. However, I think the algorithm and the quality of the data it was built upon can still be developed, especially if the helmet gets tested in a real crash simulator.

If developed further, a system of this type could reduce the time between a serious motorcycle crash and someone becoming aware that assistance may be needed, particularly when a rider is alone. However, false positives could create unnecessary alerts and false negatives could leave a genuine crash undetected. Any real deployment would therefore require much larger independent datasets, real motorcycle testing, certified helmet integration, reliable cellular communication, privacy controls and extensive safety validation before it could be trusted as an emergency system.

16.3 Future Improvements

Although this project was successful as a prototype, as mentioned previously, further development needs to be done for it to be real-world ready. In my next iteration of this helmet, I would focus on making the following improvements:

1. Test the helmet on an actual motorcycle
    a. This would help me get more realistic and comprehensive data, helping my algorithm become more accurate.
2. Build the crash detection algorithm so that it takes into account more scenarios
    a. Developing the code in this way would help improve the reliability of the product, as it would mean more potential crashes are identified.
3. Explore the use of additional sensors / components and services
    a. These components would be integrated into the crash detection system, allowing me to further advance the algorithm by taking into account factors other than just the acceleration and gyroscope.
    b. An example would be the use of a microphone to pick up surrounding sounds such as a loud crash, which would be indicative of an accident.
    c. Services or features such as geofencing could be used to alert hospitals nearby to the rider's crash location so that an ambulance can reach earlier.
4. Explore the use of a custom chip
    a. Designing and manufacturing a custom chip would greatly help reduce the size of the system, as at the moment, it is not very portable or compact. This would also remove the need for keeping all of the wires that are currently on the breadboard.
    b. Provide an option to buy a custom made crash detection safety helmet and another option to provide a retrofit that attaches easily to the previously owned helmet as a cost effective option.

17. Bibliography

1. Quin Within. (2025). *Quin Within*. [online] Available at: https://www.quinwithin.com/ [Accessed 11 Sept. 2026].
2. REALRIDER. (2025). *REALRIDER® Motorcycle Crash Detection app - keeping motorcyclists safe and connected.* [online] Available at: https://realrider.com/ [Accessed 11 Sept. 2026].
3. Davis, N. (2019). *This clever road safety app automatically calls for help within seconds of a crash*. [online] Motorcyclenews.com. Available at: https://www.motorcyclenews.com/news/new-tech/2025/october/realrider-tech/ [Accessed 11 Sept. 2026].
4. Chevincycles.com. (2026). *Tocsen, Crash Sensor for Cyclists – Chevin Cycles Hub*. [online] Available at: https://www.chevincycles.com/hub/2024/02/17/tocsen-crash-sensor-for-cyclists/ [Accessed 11 Sept. 2026].
5. Ansari, M.Y. (2026). *Simple crash detection system using Arduino*. [online] Medium. Available at: https://medium.com/@yaaminansari2727/simple-crash-detection-system-using-arduino-ff43fe81c6d9 [Accessed 13 Sept. 2026].
6. Quincy, R., Vulin, D. and Mounier, B. (1988) 'Motorcycle impacts with guardrails', Transportation Research Circular, 341, pp. 23–30. Available at: https://onlinepubs.trb.org/Onlinepubs/trcircular/341/341-003.pdf [Accessed: 5 October 2026].
7. *Lowside crash up close*. (2018). [Video] Directed by RNickeyMouse. YouTube. Available at: https://www.youtube.com/watch?v=eHBu5NMnzS8 [Accessed 13 Sept. 2026].
8. *The 4 most common motorcycle crashes & the reasons why*. (2021). [Video] Directed by Dave Moss Tuning. YouTube. Available at: https://www.youtube.com/watch?v=ddCFqFAuQ-A [Accessed 13 Sept. 2026].
9. Mips. (n.d.). *How long do helmets last?* [online] Available at: https://mipsprotection.com/helmet-safety/how-long-do-helmets-last/ [Accessed 13 Sept. 2026].
10. *ESP32 for dummies - explained in 4 minutes (super beginner-friendly!)*. (2026). [Video] Directed by Rolling Bob. YouTube. Available at: https://www.youtube.com/watch?v=6TjSI1cgijQ [Accessed 13 Sept. 2026].

11. Espressif Systems (2026). *ESP32 Series Datasheet*, Version 5.3. [online]. Available at: ESP32 Series Datasheet – Espressif Systems [Accessed 5 Oct. 2026].
12. Branch, B. (2011). *Helmet standards comparison: Snell - dot - ECE R22-05 - updated 2019*. [online] Silodrome. Available at: https://silodrome.com/snell-vs-dot-vs-ece-r22-05-helmet-standards-throwdown/ [Accessed 14 Sept. 2026].
13. https://www.driverknowledgetests.com/resources/author/darren (2014). *Where's the most likely place your helmet will take a crash impact?* [online] Driver Knowledge Test (DKT) Resources. Available at: https://www.driverknowledgetests.com/resources/wheres-the-most-likely-place-your-helmet-will-take-a-crash-impact/ [Accessed 15 Sept. 2026].
14. Department for Transport (2026). *Reported road casualties Great Britain: motorcyclist factsheet, 2025*. [online] GOV.UK. Available at: https://www.gov.uk/government/statistics/reported-road-casualties-great-britain-vulnerable-road-user-factsheets/reported-road-casualties-great-britain-motorcyclist-factsheet-2025 [Accessed 3 Oct. 2026].
15. Jackulin Mahariba, A., R., A.U. and Rajan, G.B. (2022). An efficient automatic accident detection system using inertial measurement through machine learning techniques for powered two wheelers. *Expert Systems with Applications*, 192, p.116389. doi:10.1016/j.eswa.2021.116389.
16. Khalid, W., Jamil, M., Ashraf Ali Khan and Qasim Awais (2024). Open-Source Internet of Things-Based Supervisory Control and Data Acquisition System for Photovoltaic Monitoring and Control Using HTTP and TCP/IP Protocols. *Energies*, 17(16), pp.4083–4083. doi:10.3390/en17164083.
17. InvenSense Inc. (2013). *MPU-6000 and MPU-6050 Product Specification*, Revision 3.4, Document No. PS-MPU-6000A-00. [online]. Available at: Official MPU-6000/MPU-6050 Product Specification [Accessed 5 Oct. 2026].
18. Blynk Technologies Inc. (2026). *Hardware and Connectivity*. [online]. Available at: Blynk Hardware and Connectivity [Accessed 5 Oct. 2026].
19. Blynk Technologies Inc. (2021). *How to get started with the new Blynk*. [online]. Available at: How to get started with Blynk [Accessed 5 Oct. 2026].
20. u-blox AG (2011). *NEO-6 u-blox 6 GPS Modules Data Sheet*, Document No. GPS.G6-HW-09005. [online]. Available at: Official NEO-6 Data Sheet [Accessed 5 Oct. 2026].